

# BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova  
Google AI Language  
{jacobdevlin, mingweichang, kentonl, kristout}@google.com

## Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

## 1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as named entity recognition and question answering, where models are required to produce fine-grained output at the token level (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **Bidirectional Encoder Representations from Transformers**. BERT alleviates the previously mentioned unidirectionality constraint by using a “masked language model” (MLM) pre-training objective, inspired by the Cloze task (Taylor, 1953). The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked

# BERT：用于语言理解的深度双向Transformer预训练

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina ToutanovaGoogle  
AI 语言{jacobdevlin, mingweichang, kentonl, kristout}@google.com

## 摘要

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT 在概念上简单，而在实证上强大。它在十一项自然语言处理任务上取得了新的最先进 (state-of-the-art) 结果，包括将 GLUE 得分提升至 80.5% (绝对提升 7.7 个百分点)，MultiNLI 准确率提升至 86.7% (绝对提升 4.6%)，SQuAD v1.1 问答 Test F1 提升至 93.2 (绝对提升 1.5 分)，以及 SQuAD v2.0 Test F1 提升至 83.1 (绝对提升 5.1 分)。

## 1 引言

语言模型预训练已被证明对改善许多自然语言处理任务有效 (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018)。这些任务包括句子级任务，如自然语言推理 (Bowman et al., 2015; Williams et al., 2018) 和释义 (Dolan and Brockett, 2005)，这类任务通过整体分析句子来预测句子间的关系；还包括词元级任务，如命名实体识别和问答，这类任务要求模型在词元级别产生细粒度的输出 (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016)。

将预训练语言表示应用于下游任务有两种现有策略：基于特征的方式和微调方式。基于特征的方法，如ELMo (Peters et al., 2018a)，使用特定于任务的架构，将预训练表示作为额外特征纳入其中。微调方法，如生成式预训练变换器 (OpenAIGPT) (Radford et al., 2018)，引入最少量的任务特定参数，并通过简单地微调所有预训练参数来在下游任务上进行训练。两种方法在预训练期间共享相同的目标函数，它们都使用单向语言模型来学习通用语言表示。

我们认为，当前的技术限制了预训练表示的能力，尤其是对于微调方法而言。主要的局限性在于标准语言模型是单向的，这限制了预训练期间可采用的架构选择。例如，在OpenAI GPT中，作者使用了从左到右的架构，在Transformer的自注意力层中，每个token只能关注其之前的token (Vaswani et al., 2017)。这种限制对于句子级任务来说是次优的，而当将基于微调的方法应用于问答等token级任务时，其危害可能非常大，因为在此类任务中，融合来自两个方向的上下文至关重要。

在本文中，我们通过提出BERT来改进基于微调的方法：**Bidirectional Encoder Representations from Transformers**。BERT通过采用“掩码语言模型” (MLM) 预训练目标来缓解前文提到的单向性约束，该目标受完形填空 (Cloze) 任务的启发 (Taylor, 1953)。掩码语言模型会随机遮蔽输入中的一些词符，其目标是预测被遮蔽词符的原始词汇表 id

word based only on its context. Unlike left-to-right language model pre-training, the MLM objective enables the representation to fuse the left and the right context, which allows us to pre-train a deep bidirectional Transformer. In addition to the masked language model, we also use a “next sentence prediction” task that jointly pre-trains text-pair representations. The contributions of our paper are as follows:

- We demonstrate the importance of bidirectional pre-training for language representations. Unlike Radford et al. (2018), which uses unidirectional language models for pre-training, BERT uses masked language models to enable pre-trained deep bidirectional representations. This is also in contrast to Peters et al. (2018a), which uses a shallow concatenation of independently trained left-to-right and right-to-left LMs.
- We show that pre-trained representations reduce the need for many heavily-engineered task-specific architectures. BERT is the first fine-tuning based representation model that achieves state-of-the-art performance on a large suite of sentence-level and token-level tasks, outperforming many task-specific architectures.
- BERT advances the state of the art for eleven NLP tasks. The code and pre-trained models are available at <https://github.com/google-research/bert>.

## 2 Related Work

There is a long history of pre-training general language representations, and we briefly review the most widely-used approaches in this section.

### 2.1 Unsupervised Feature-based Approaches

Learning widely applicable representations of words has been an active area of research for decades, including non-neural (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) and neural (Mikolov et al., 2013; Pennington et al., 2014) methods. Pre-trained word embeddings are an integral part of modern NLP systems, offering significant improvements over embeddings learned from scratch (Turian et al., 2010). To pre-train word embedding vectors, left-to-right language modeling objectives have been used (Mnih and Hinton, 2009), as well as objectives to discriminate correct from incorrect words in left and right context (Mikolov et al., 2013).

These approaches have been generalized to coarser granularities, such as sentence embeddings (Kiros et al., 2015; Logeswaran and Lee, 2018) or paragraph embeddings (Le and Mikolov, 2014). To train sentence representations, prior work has used objectives to rank candidate next sentences (Jernite et al., 2017; Logeswaran and Lee, 2018), left-to-right generation of next sentence words given a representation of the previous sentence (Kiros et al., 2015), or denoising auto-encoder derived objectives (Hill et al., 2016).

ELMo and its predecessor (Peters et al., 2017, 2018a) generalize traditional word embedding research along a different dimension. They extract *context-sensitive* features from a left-to-right and a right-to-left language model. The contextual representation of each token is the concatenation of the left-to-right and right-to-left representations. When integrating contextual word embeddings with existing task-specific architectures, ELMo advances the state of the art for several major NLP benchmarks (Peters et al., 2018a) including question answering (Rajpurkar et al., 2016), sentiment analysis (Socher et al., 2013), and named entity recognition (Tjong Kim Sang and De Meulder, 2003). Melamud et al. (2016) proposed learning contextual representations through a task to predict a single word from both left and right context using LSTMs. Similar to ELMo, their model is feature-based and not deeply bidirectional. Fedus et al. (2018) shows that the cloze task can be used to improve the robustness of text generation models.

### 2.2 Unsupervised Fine-tuning Approaches

As with the feature-based approaches, the first works in this direction only pre-trained word embedding parameters from unlabeled text (Collobert and Weston, 2008).

More recently, sentence or document encoders which produce contextual token representations have been pre-trained from unlabeled text and fine-tuned for a supervised downstream task (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018). The advantage of these approaches is that few parameters need to be learned from scratch. At least partly due to this advantage, OpenAI GPT (Radford et al., 2018) achieved previously state-of-the-art results on many sentence-level tasks from the GLUE benchmark (Wang et al., 2018a). Left-to-right language model-

仅根据其上下文来预测该词。与从左到右的语言模型预训练不同，MLM 目标使表示能够融合左侧和右侧的上下文，这让我们能够预训练一个深度双向 Transformer。除了掩码语言模型之外，我们还使用“下一句预测”任务来联合预训练文本对的表示。我们论文的贡献如下：

- 我们证明了双向预训练对语言表示的重要性。不同于 Radford 等人 (2018)，其使用单向语言模型进行预训练，而 BERT 使用掩码语言模型来实现预训练的深度双向表示。这也不同于 Peters 等人 (2018a)，后者使用独立训练的从左到右和从右到左语言模型的浅层拼接。
- 我们证明，预训练表示降低了对许多高度工程化的任务特定架构的需求。BERT 是首个基于微调的表示模型，它在大量句子级和词元级任务上取得了最先进的性能，优于许多任务特定架构。

- BERT 在 11 项 NLP 任务中取得了当时最先进的成果。代码和预训练模型可在以下网址获取：<https://github.com/google-research/bert>。

## 2 相关工作

预训练通用语言表征有着悠久的历史，我们在本节中简要回顾最广泛使用的方法。

### 2.1 基于特征的无监督方法

学习具有广泛适用性的词表示数十年来一直是一个活跃的研究领域，包括非神经 (Brown 等人, 1992; Ando 和 Zhang, 2005; Blitzer 等人, 2006) 和神经 (Mikolov 等人, 2013; Pennington 等人, 2014) 方法。预训练词嵌入是现代 NLP 系统不可或缺的组成部分，与从头学习的嵌入相比能带来显著提升 (Turian 等人, 2010)。为了预训练词嵌入向量，人们使用了从左到右的语言建模目标 (Mnih 和 Hinton, 2009)，以及用于在左右上下文中区分正确词与错误词的目标 (Mikolov 等人, 2013)。

These approaches have been generalized to coarser granularities, such as sentence embeddings (Kiros et al., 2015; Logeswaran and Lee, 2018) or paragraph embeddings (Le and Mikolov, 2014). To train sentence representations, prior work has used objectives to rank candidate next sentences (Jernite et al., 2017; Logeswaran and Lee, 2018), left-to-right generation of next sentence words given a representation of the previous sentence (Kiros et al., 2015), or denoising auto-encoder derived objectives (Hill et al., 2016).

ELMo and its predecessor (Peters et al., 2017, 2018a) generalize traditional word embedding research along a different dimension. They extract *context-sensitive* features from a left-to-right and a right-to-left language model. The contextual representation of each token is the concatenation of the left-to-right and right-to-left representations. When integrating contextual word embeddings with existing task-specific architectures, ELMo advances the state of the art for several major NLP benchmarks (Peters et al., 2018a) including question answering (Rajpurkar et al., 2016), sentiment analysis (Socher et al., 2013), and named entity recognition (Tjong Kim Sang and De Meulder, 2003). Melamud et al. (2016) proposed learning contextual representations through a task to predict a single word from both left and right context using LSTMs. Similar to ELMo, their model is feature-based and not deeply bidirectional. Fedus et al. (2018) shows that the cloze task can be used to improve the robustness of text generation models.

### 2.2 无监督微调方法

与基于特征的方法类似，该方向的早期工作仅从无标注文本中预训练词嵌入参数 (Collobert 和 Weston, 2008)。

More recently, sentence or document encoders which produce contextual token representations have been pre-trained from unlabeled text and fine-tuned for a supervised downstream task (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018). The advantage of these approaches is that few parameters need to be learned from scratch. At least partly due to this advantage, OpenAI GPT (Radford et al., 2018) achieved previously state-of-the-art results on many sentence-level tasks from the GLUE benchmark (Wang et al., 2018a). Left-to-right language model-

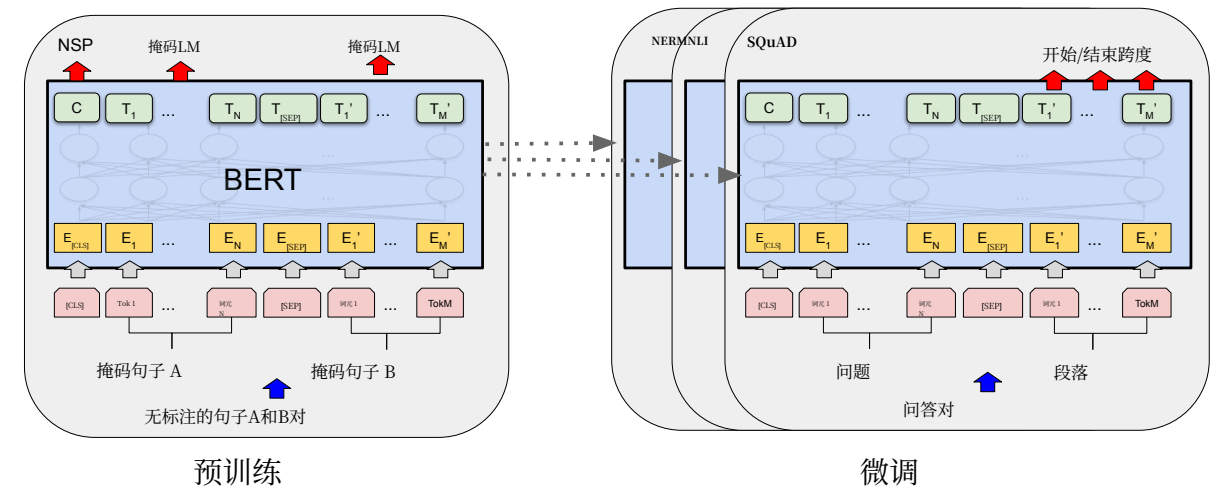
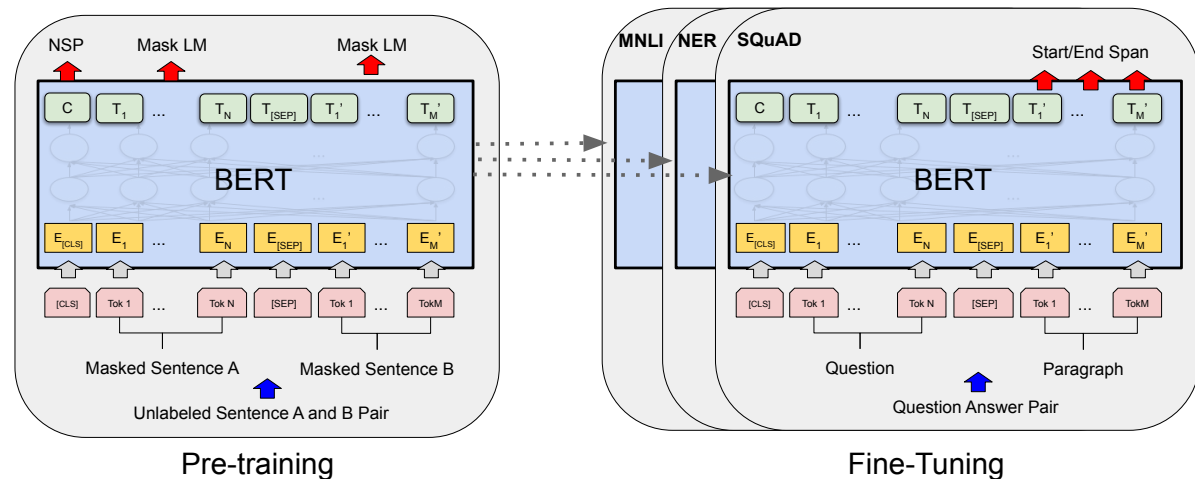


图1: BERT 的整体预训练与微调流程。除输出层外，预训练和微调均使用相同的架构。相同的预训练模型参数用于初始化不同下游任务的模型。在微调过程中，所有参数都会被微调。[CLS] 是添加在每个输入示例前面的特殊符号，而 [SEP] 是特殊的分隔符标记（例如用于分隔问题/答案）。

ing 和自编码器目标已被用于预训练此类模型（Howard 和 Ruder, 2018; Radford 等人, 2018; Dai 和 Le, 2015）。

### 2.3 从有监督数据中进行迁移学习

也有研究工作表明，从带有大规模数据集的有监督任务中可以进行有效的迁移，例如自然语言推理（Conneau 等人, 2017）和机器翻译（McCann 等人, 2017）。计算机视觉领域的研究也证明了从大规模预训练模型进行迁移学习的重要性，其中一种有效的做法是对使用 ImageNet 预训练的模型进行微调（Deng 等人, 2009; Yosinski 等人, 2014）。

## 3 BERT

本节将介绍 BERT 及其详细实现。我们的框架包含两个步骤：预训练和微调。在预训练阶段，模型在无标注数据上针对不同的预训练任务进行训练。在微调阶段，BERT 模型首先用预训练参数进行初始化，然后使用来自下游任务的标注数据对所有参数进行微调。尽管都用相同的预训练参数进行初始化，但每个下游任务都有各自独立的微调模型。图 1 中的问答示例将作为本节的贯穿示例。

BERT 的一个显著特点是它不同任务间使用统一的架构。仅有极小的

预训练架构与最终下游架构之间的差异。

**模型架构** BERT 的模型架构是一种多层双向 Transformer 编码器，基于 Vaswani 等人（2017）所描述的原始实现，并在 tensor2tensor 库中发布。<sup>1</sup> 由于 Transformer 的使用已经十分普遍，且我们的实现与原始实现几乎完全相同，因此我们将省略对模型架构的详尽背景介绍，读者可参阅 Vaswani 等人（2017）以及《The Annotated Transformer》等优秀指南。<sup>2</sup>

在本工作中，我们将层数（即 Transformer 块）记为  $L$ ，隐藏层大小记为  $H$ ，自注意力头数记为  $A$ 。<sup>3</sup> 我们主要报告两种模型规模的结果：BERT<sub>BASE</sub> ( $L=12$ ,  $H=768$ ,  $A=12$ , 总参数量=110M) 和 BERT<sub>LARGE</sub> ( $L=24$ ,  $H=1024$ ,  $A=16$ , 总参数量=340M)。

BERT<sub>BASE</sub> 被选择为具有与 OpenAI GPT 相同的模型规模，以便进行比较。然而，至关重要是，BERT Transformer 使用双向自注意力机制，而 GPT Transformer 使用受限自注意力机制，即每个 token 只能关注其左侧的上下文。<sup>4</sup>

<sup>1</sup><https://github.com/tensorflow/tensor2tensor>  
<sup>2</sup><http://nlp.seas.harvard.edu/2018/04/03/attention.html><sup>3</sup>在所有情况下，我们都将前馈/滤波器大小设为  $4H$ ，即  $H=768$  为 3072， $H=1024$  为 4096。<sup>4</sup>我们注意到，在文献中，双向 Trans-

ing and auto-encoder objectives have been used for pre-training such models (Howard and Ruder, 2018; Radford et al., 2018; Dai and Le, 2015).

### 2.3 Transfer Learning from Supervised Data

There has also been work showing effective transfer from supervised tasks with large datasets, such as natural language inference (Conneau et al., 2017) and machine translation (McCann et al., 2017). Computer vision research has also demonstrated the importance of transfer learning from large pre-trained models, where an effective recipe is to fine-tune models pre-trained with ImageNet (Deng et al., 2009; Yosinski et al., 2014).

## 3 BERT

We introduce BERT and its detailed implementation in this section. There are two steps in our framework: *pre-training* and *fine-tuning*. During pre-training, the model is trained on unlabeled data over different pre-training tasks. For fine-tuning, the BERT model is first initialized with the pre-trained parameters, and all of the parameters are fine-tuned using labeled data from the downstream tasks. Each downstream task has separate fine-tuned models, even though they are initialized with the same pre-trained parameters. The question-answering example in Figure 1 will serve as a running example for this section.

A distinctive feature of BERT is its unified architecture across different tasks. There is mini-

mal difference between the pre-trained architecture and the final downstream architecture.

**Model Architecture** BERT’s model architecture is a multi-layer bidirectional Transformer encoder based on the original implementation described in Vaswani et al. (2017) and released in the tensor2tensor library.<sup>1</sup> Because the use of Transformers has become common and our implementation is almost identical to the original, we will omit an exhaustive background description of the model architecture and refer readers to Vaswani et al. (2017) as well as excellent guides such as “The Annotated Transformer.”<sup>2</sup>

In this work, we denote the number of layers (i.e., Transformer blocks) as  $L$ , the hidden size as  $H$ , and the number of self-attention heads as  $A$ .<sup>3</sup> We primarily report results on two model sizes: BERT<sub>BASE</sub> ( $L=12$ ,  $H=768$ ,  $A=12$ , Total Parameters=110M) and BERT<sub>LARGE</sub> ( $L=24$ ,  $H=1024$ ,  $A=16$ , Total Parameters=340M).

BERT<sub>BASE</sub> was chosen to have the same model size as OpenAI GPT for comparison purposes. Critically, however, the BERT Transformer uses bidirectional self-attention, while the GPT Transformer uses constrained self-attention where every token can only attend to context to its left.<sup>4</sup>

<sup>1</sup><https://github.com/tensorflow/tensor2tensor>

<sup>2</sup><http://nlp.seas.harvard.edu/2018/04/03/attention.html>

<sup>3</sup>In all cases we set the feed-forward/filter size to be  $4H$ , i.e., 3072 for the  $H=768$  and 4096 for the  $H=1024$ .

<sup>4</sup>We note that in the literature the bidirectional Trans-

**Input/Output Representations** To make BERT handle a variety of down-stream tasks, our input representation is able to unambiguously represent both a single sentence and a pair of sentences (e.g.,  $\langle \text{Question, Answer} \rangle$ ) in one token sequence. Throughout this work, a “sentence” can be an arbitrary span of contiguous text, rather than an actual linguistic sentence. A “sequence” refers to the input token sequence to BERT, which may be a single sentence or two sentences packed together.

We use WordPiece embeddings (Wu et al., 2016) with a 30,000 token vocabulary. The first token of every sequence is always a special classification token ( $[\text{CLS}]$ ). The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence. We differentiate the sentences in two ways. First, we separate them with a special token ( $[\text{SEP}]$ ). Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B. As shown in Figure 1, we denote input embedding as  $E$ , the final hidden vector of the special  $[\text{CLS}]$  token as  $C \in \mathbb{R}^H$ , and the final hidden vector for the  $i^{\text{th}}$  input token as  $T_i \in \mathbb{R}^H$ .

For a given token, its input representation is constructed by summing the corresponding token, segment, and position embeddings. A visualization of this construction can be seen in Figure 2.

### 3.1 Pre-training BERT

Unlike Peters et al. (2018a) and Radford et al. (2018), we do not use traditional left-to-right or right-to-left language models to pre-train BERT. Instead, we pre-train BERT using two unsupervised tasks, described in this section. This step is presented in the left part of Figure 1.

**Task #1: Masked LM** Intuitively, it is reasonable to believe that a deep bidirectional model is strictly more powerful than either a left-to-right model or the shallow concatenation of a left-to-right and a right-to-left model. Unfortunately, standard conditional language models can only be trained left-to-right *or* right-to-left, since bidirectional conditioning would allow each word to indirectly “see itself”, and the model could trivially predict the target word in a multi-layered context.

former is often referred to as a “Transformer encoder” while the left-context-only version is referred to as a “Transformer decoder” since it can be used for text generation.

In order to train a deep bidirectional representation, we simply mask some percentage of the input tokens at random, and then predict those masked tokens. We refer to this procedure as a “masked LM” (MLM), although it is often referred to as a *Cloze* task in the literature (Taylor, 1953). In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random. In contrast to denoising auto-encoders (Vincent et al., 2008), we only predict the masked words rather than reconstructing the entire input.

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the  $[\text{MASK}]$  token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual  $[\text{MASK}]$  token. The training data generator chooses 15% of the token positions at random for prediction. If the  $i$ -th token is chosen, we replace the  $i$ -th token with (1) the  $[\text{MASK}]$  token 80% of the time (2) a random token 10% of the time (3) the unchanged  $i$ -th token 10% of the time. Then,  $T_i$  will be used to predict the original token with cross entropy loss. We compare variations of this procedure in Appendix C.2.

### Task #2: Next Sentence Prediction (NSP)

Many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) are based on understanding the *relationship* between two sentences, which is not directly captured by language modeling. In order to train a model that understands sentence relationships, we pre-train for a binarized *next sentence prediction* task that can be trivially generated from any monolingual corpus. Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labeled as  $\text{IsNext}$ ), and 50% of the time it is a random sentence from the corpus (labeled as  $\text{NotNext}$ ). As we show in Figure 1,  $C$  is used for next sentence prediction (NSP).<sup>5</sup> Despite its simplicity, we demonstrate in Section 5.1 that pre-training towards this task is very beneficial to both QA and NLI.<sup>6</sup>

<sup>5</sup>The final model achieves 97%-98% accuracy on NSP.

<sup>6</sup>The vector  $C$  is not a meaningful sentence representation without fine-tuning, since it was trained with NSP.

**输入/输出表示** 为了使 BERT 能够处理各种下游任务，我们的输入表示能够在同一个词元序列中无歧义地表示单个句子和句子对（例如  $\langle \text{问题, 回答} \rangle$ ）。在整项工作中，“句子”可以是连续文本的任意跨度，而不一定是真正意义上的语言学句子。“序列”指的是输入到 BERT 的词元序列，它可以是一个单独的句子，也可以是打包在一起的两个句子。

We use WordPiece embeddings (Wu et al., 2016) with a 30,000 token vocabulary. The first token of every sequence is always a special classification token ( $[\text{CLS}]$ ). The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence. We differentiate the sentences in two ways. First, we separate them with a special token ( $[\text{SEP}]$ ). Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B. As shown in Figure 1, we denote input embedding as  $E$ , the final hidden vector of the special  $[\text{CLS}]$  token as  $C \in \mathbb{R}^H$ , and the final hidden vector for the  $i^{\text{th}}$  input token as  $T_i \in \mathbb{R}^H$ .

对于给定的词元，其输入表示是通过将相应的词元嵌入、片段嵌入和位置嵌入相加而构造得到的。这一构造过程的可视化见图2。

### 3.1 BERT预训练

不同于 Peters 等人 (2018a) 和 Radford 等人 (2018)，我们不使用传统的从左到右或从右到左的语言模型来预训练 BERT。相反，我们使用两个无监督任务来预训练 BERT，这两个任务将在本节中描述。该步骤展示于图1 的左半部分。

**任务#1：掩码语言模型（Masked LM）** 直观上，我们有理由认为，深度双向模型严格强于从左到右模型，也严格强于从左到右模型与从右到左模型的浅层拼接。遗憾的是，标准的条件语言模型只能从左到右或从右到左进行训练，因为双向条件化会使每个词间接地“看到自己”，从而模型就能在多层上下文中轻而易举地预测出目标词。

前者通常被称为“Transformer 编码器”，而仅依赖左侧上下文的版本则被称为“Transformer 解码器”，因为它可用于文本生成。

为了训练深度双向表示，我们只需随机遮蔽一定百分比的输入词符，然后预测这些被遮蔽的词符。我们将这一过程称为“掩码语言模型”

(MLM)，尽管在文献中它通常被称为完形填空任务 (Taylor, 1953)。在这种情况下，与被遮蔽词符对应的最终隐藏向量会被送入一个基于词表的输出 softmax，如同标准的语言模型一样。在我们所有的实验中，我们随机遮蔽每个序列中 15% 的 WordPiece 词符。与去噪自编码器 (Vincent 等, 2008) 不同，我们只预测被遮蔽的词，而不是重构整个输入。

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the  $[\text{MASK}]$  token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual  $[\text{MASK}]$  token. The training data generator chooses 15% of the token positions at random for prediction. If the  $i$ -th token is chosen, we replace the  $i$ -th token with (1) the  $[\text{MASK}]$  token 80% of the time (2) a random token 10% of the time (3) the unchanged  $i$ -th token 10% of the time. Then,  $T_i$  will be used to predict the original token with cross entropy loss. We compare variations of this procedure in Appendix C.2.

### Task #2: Next Sentence Prediction (NSP)

Many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) are based on understanding the *relationship* between two sentences, which is not directly captured by language modeling. In order to train a model that understands sentence relationships, we pre-train for a binarized *next sentence prediction* task that can be trivially generated from any monolingual corpus. Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labeled as  $\text{IsNext}$ ), and 50% of the time it is a random sentence from the corpus (labeled as  $\text{NotNext}$ ). As we show in Figure 1,  $C$  is used for next sentence prediction (NSP).<sup>5</sup> Despite its simplicity, we demonstrate in Section 5.1 that pre-training towards this task is very beneficial to both QA and NLI.<sup>6</sup>

<sup>5</sup>最终模型在 NSP 任务上达到了 97%-98% 的准确率。“向量  $C$  在未经微调的情况下并不是一个有意义的句子表示，因为它通过 NSP 训练得到的。

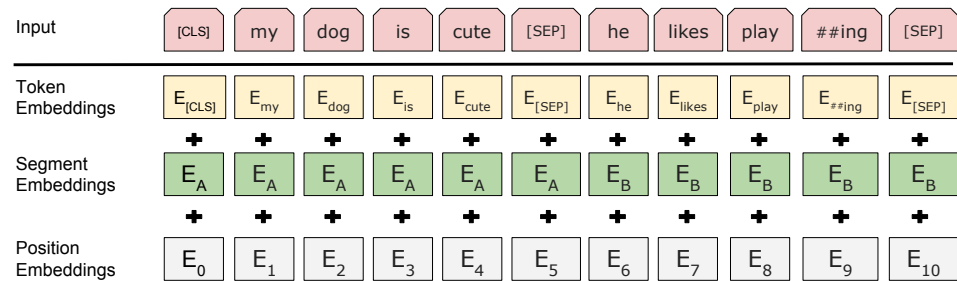


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The NSP task is closely related to representation-learning objectives used in [Jernite et al. \(2017\)](#) and [Logeswaran and Lee \(2018\)](#). However, in prior work, only sentence embeddings are transferred to down-stream tasks, where BERT transfers all parameters to initialize end-task model parameters.

**Pre-training data** The pre-training procedure largely follows the existing literature on language model pre-training. For the pre-training corpus we use the BooksCorpus (800M words) ([Zhu et al., 2015](#)) and English Wikipedia (2,500M words). For Wikipedia we extract only the text passages and ignore lists, tables, and headers. It is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark ([Chelba et al., 2013](#)) in order to extract long contiguous sequences.

### 3.2 Fine-tuning BERT

Fine-tuning is straightforward since the self-attention mechanism in the Transformer allows BERT to model many downstream tasks—whether they involve single text or text pairs—by swapping out the appropriate inputs and outputs. For applications involving text pairs, a common pattern is to independently encode text pairs before applying bidirectional cross attention, such as [Parikh et al. \(2016\)](#); [Seo et al. \(2017\)](#). BERT instead uses the self-attention mechanism to unify these two stages, as encoding a concatenated text pair with self-attention effectively includes *bidirectional* cross attention between two sentences.

For each task, we simply plug in the task-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end. At the input, sentence A and sentence B from pre-training are analogous to (1) sentence pairs in paraphrasing, (2) hypothesis-premise pairs in entailment, (3) question-passage pairs in question answering, and

(4) a degenerate text- $\emptyset$  pair in text classification or sequence tagging. At the output, the token representations are fed into an output layer for token-level tasks, such as sequence tagging or question answering, and the [CLS] representation is fed into an output layer for classification, such as entailment or sentiment analysis.

Compared to pre-training, fine-tuning is relatively inexpensive. All of the results in the paper can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the exact same pre-trained model.<sup>7</sup> We describe the task-specific details in the corresponding subsections of Section 4. More details can be found in Appendix A.5.

## 4 Experiments

In this section, we present BERT fine-tuning results on 11 NLP tasks.

### 4.1 GLUE

The General Language Understanding Evaluation (GLUE) benchmark ([Wang et al., 2018a](#)) is a collection of diverse natural language understanding tasks. Detailed descriptions of GLUE datasets are included in Appendix B.1.

To fine-tune on GLUE, we represent the input sequence (for single sentence or sentence pairs) as described in Section 3, and use the final hidden vector  $C \in \mathbb{R}^H$  corresponding to the first input token ([CLS]) as the aggregate representation. The only new parameters introduced during fine-tuning are classification layer weights  $W \in \mathbb{R}^{K \times H}$ , where  $K$  is the number of labels. We compute a standard classification loss with  $C$  and  $W$ , i.e.,  $\log(\text{softmax}(CW^T))$ .

<sup>7</sup>For example, the BERT SQuAD model can be trained in around 30 minutes on a single Cloud TPU to achieve a Dev F1 score of 91.0%.

<sup>8</sup>See (10) in <https://gluebenchmark.com/faq>.

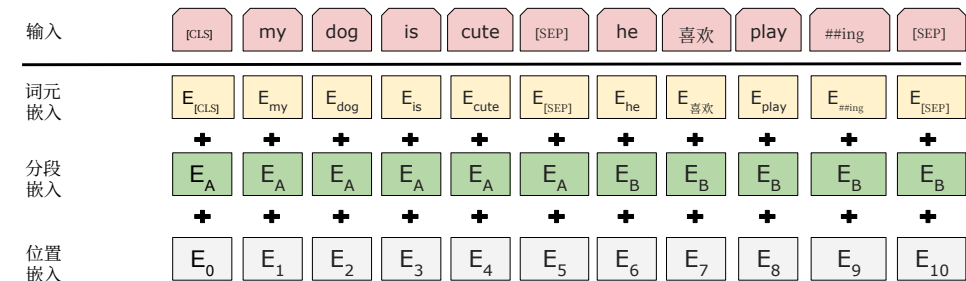


图 2: BERT 的输入表示。输入嵌入是词元嵌入、分段嵌入和位置嵌入之和。

NSP 任务与以下工作中使用的表示学习目标密切相关: [Jernite 等人 \(2017\)](#) 以及 [Logeswaran 和 Lee \(2018\)](#)。然而, 在先前的工作中, 仅有句子嵌入被迁移到下游任务, 而 BERT 则会迁移所有参数以初始化最终任务的模型参数。

**预训练数据** 预训练过程在很大程度上遵循现有的语言模型预训练文献。对于预训练语料库, 我们使用 BooksCorpus (8 亿词) ([Zhu 等人, 2015](#)) 以及英文维基百科 (25 亿词)。对于维基百科, 我们仅提取文本段落, 忽略列表、表格和标题。至关重要的是, 应使用文档级语料库, 而非诸如十亿词基准 (Billion Word Benchmark) ([Chelba 等人, 2013](#)) 这样打乱顺序的句子级语料库, 以便提取长连续序列。

### 3.2 微调 BERT

微调非常直接, 因为 Transformer 中的自注意力机制允许 BERT 对许多下游任务进行建模——无论这些任务涉及单个文本还是文本对——只需替换相应的输入和输出即可。对于涉及文本对的应用, 一种常见的模式是在应用双向交叉注意力之前先独立编码文本对, 例如 [Parikh 等人 \(2016\)](#); [Seo 等人 \(2017\)](#)。而 BERT 则使用自注意力机制将这两个阶段统一起来, 因为使用自注意力对拼接后的文本对进行编码, 实际上已经有效地包含了两个句子之间的双向交叉注意力。

对于每个任务, 我们只需将任务特定的输入和输出插入 BERT, 并对所有参数进行端到端的微调。在输入端, 预训练中的句子 A 和句子 B 类似于 (1) 释义任务中的句子对、(2) 蕴含任务中的假设-前提对、(3) 问答任务中的问题-段落对, 以及

(4) 文本分类或序列标注中的退化的文本- $\emptyset$  对。在输出端, 词元表示被送入用于词元级任务 (如序列标注或问答) 的输出层, 而 [CLS] 的表示则被送入用于分类任务 (如蕴含判断或情感分析) 的输出层。

与预训练相比, 微调的开销相对较小。从完全相同的预训练模型出发, 论文中的所有结果都可以在单个 Cloud TPU 上至多 1 小时内复现, 或在 GPU 上花几个小时复现。<sup>7</sup> 我们在第 4 节的相应小节中描述任务特定的细节。更多细节见附录 A.5。

## 4 实验

在本节中, 我们展示了 BERT 在 11 个 NLP 任务上的微调结果。

### 4.1 GLUE

通用语言理解评估 (GLUE) 基准 ([Wang 等人, 2018a](#)) 是多种多样自然语言理解任务的集合。GLUE 数据集的详细描述见附录 B.1。

To fine-tune on GLUE, we represent the input sequence (for single sentence or sentence pairs) as described in Section 3, and use the final hidden vector  $C \in \mathbb{R}^H$  corresponding to the first input token ([CLS]) as the aggregate representation. The only new parameters introduced during fine-tuning are classification layer weights  $W \in \mathbb{R}^{K \times H}$ , where  $K$  is the number of labels. We compute a standard classification loss with  $C$  and  $W$ , i.e.,  $\log(\text{softmax}(CW^T))$ .

<sup>7</sup>例如, BERT SQuAD 模型在单个 Cloud TPU 上训练约 30 分钟即可达到 91.0% 的开发集 F1 分数。<sup>8</sup>参见 (10), 详见 <https://gluebenchmark.com/faq>。

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average<br>- |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|--------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0         |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0         |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1         |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6         |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b>  |

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The ‘‘Average’’ column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.<sup>8</sup> BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

We use a batch size of 32 and fine-tune for 3 epochs over the data for all GLUE tasks. For each task, we selected the best fine-tuning learning rate (among 5e-5, 4e-5, 3e-5, and 2e-5) on the Dev set. Additionally, for BERT<sub>LARGE</sub> we found that fine-tuning was sometimes unstable on small datasets, so we ran several random restarts and selected the best model on the Dev set. With random restarts, we use the same pre-trained checkpoint but perform different fine-tuning data shuffling and classifier layer initialization.<sup>9</sup>

Results are presented in Table 1. Both BERT<sub>BASE</sub> and BERT<sub>LARGE</sub> outperform all systems on all tasks by a substantial margin, obtaining 4.5% and 7.0% respective average accuracy improvement over the prior state of the art. Note that BERT<sub>BASE</sub> and OpenAI GPT are nearly identical in terms of model architecture apart from the attention masking. For the largest and most widely reported GLUE task, MNLI, BERT obtains a 4.6% absolute accuracy improvement. On the official GLUE leaderboard<sup>10</sup>, BERT<sub>LARGE</sub> obtains a score of 80.5, compared to OpenAI GPT, which obtains 72.8 as of the date of writing.

We find that BERT<sub>LARGE</sub> significantly outperforms BERT<sub>BASE</sub> across all tasks, especially those with very little training data. The effect of model size is explored more thoroughly in Section 5.2.

## 4.2 SQuAD v1.1

The Stanford Question Answering Dataset (SQuAD v1.1) is a collection of 100k crowd-sourced question/answer pairs (Rajpurkar et al., 2016). Given a question and a passage from

<sup>9</sup>The GLUE data set distribution does not include the Test labels, and we only made a single GLUE evaluation server submission for each of BERT<sub>BASE</sub> and BERT<sub>LARGE</sub>.

<sup>10</sup><https://gluebenchmark.com/leaderboard>

Wikipedia containing the answer, the task is to predict the answer text span in the passage.

As shown in Figure 1, in the question answering task, we represent the input question and passage as a single packed sequence, with the question using the A embedding and the passage using the B embedding. We only introduce a start vector  $S \in \mathbb{R}^H$  and an end vector  $E \in \mathbb{R}^H$  during fine-tuning. The probability of word  $i$  being the start of the answer span is computed as a dot product between  $T_i$  and  $S$  followed by a softmax over all of the words in the paragraph:  $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$ . The analogous formula is used for the end of the answer span. The score of a candidate span from position  $i$  to position  $j$  is defined as  $S \cdot T_i + E \cdot T_j$ , and the maximum scoring span where  $j \geq i$  is used as a prediction. The training objective is the sum of the log-likelihoods of the correct start and end positions. We fine-tune for 3 epochs with a learning rate of 5e-5 and a batch size of 32.

Table 2 shows top leaderboard entries as well as results from top published systems (Seo et al., 2017; Clark and Gardner, 2018; Peters et al., 2018a; Hu et al., 2018). The top results from the SQuAD leaderboard do not have up-to-date public system descriptions available,<sup>11</sup> and are allowed to use any public data when training their systems. We therefore use modest data augmentation in our system by first fine-tuning on TriviaQA (Joshi et al., 2017) before fine-tuning on SQuAD.

Our best performing system outperforms the top leaderboard system by +1.5 F1 in ensembling and +1.3 F1 as a single system. In fact, our single BERT model outperforms the top ensemble system in terms of F1 score. Without TriviaQA fine-

<sup>11</sup>QANet is described in Yu et al. (2018), but the system has improved substantially after publication.

| 系统                    | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | 平均值<br>-    |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|-------------|
| OpenAI 之前的 SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0        |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0        |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1        |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6        |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b> |

表1: GLUE测试结果, 由评测服务器 (<https://gluebenchmark.com/leaderboard>) 评分。每个任务下方的数字表示训练样例的数量。‘‘平均’’一列与官方GLUE得分略有不同, 因为我们排除了存在问题的WNLI数据集。<sup>8</sup> BERT和OpenAI GPT为单模型、单任务。QQP和MRPC报告的是F1分数, STS-B报告的是Spearman相关系数, 其他任务报告的是准确率分数。我们排除了将BERT作为其组件之一的条目。

我们对所有 GLUE 任务使用 32 的批大小, 并在数据上进行 3 个 epoch 的微调。对于每个任务, 我们在开发集上选出了最佳的微调学习率 (在 5e-5、4e-5、3e-5 和 2e-5 之中选取)。此外, 对于 BERT<sub>LARGE</sub>, 我们发现微调在小数据集上有时并不稳定, 因此我们进行了多次随机重启, 并在开发集上选出最佳模型。在随机重启时, 我们使用相同的预训练检查点, 但采用不同的微调数据打乱方式和分类器层初始化。<sup>9</sup>

Results are presented in Table 1. Both BERT<sub>BASE</sub> and BERT<sub>LARGE</sub> outperform all systems on all tasks by a substantial margin, obtaining 4.5% and 7.0% respective average accuracy improvement over the prior state of the art. Note that BERT<sub>BASE</sub> and OpenAI GPT are nearly identical in terms of model architecture apart from the attention masking. For the largest and most widely reported GLUE task, MNLI, BERT obtains a 4.6% absolute accuracy improvement. On the official GLUE leaderboard<sup>10</sup>, BERT<sub>LARGE</sub> obtains a score of 80.5, compared to OpenAI GPT, which obtains 72.8 as of the date of writing.

We find that BERT<sub>LARGE</sub> significantly outperforms BERT<sub>BASE</sub> across all tasks, especially those with very little training data. The effect of model size is explored more thoroughly in Section 5.2.

## 4.2 SQuAD v1.1

斯坦福问答数据集 (SQuAD v1.1) 是一个包含 10 万个众包问答对的集合 (Rajpurkar 等人, 2016)。给定一个问题以及一段来自

<sup>9</sup>GLUE 数据集的分发不包含测试集 (Test) 标签, 我们仅为 BERT<sub>BASE</sub> 和 BERT<sub>LARGE</sub>各自向 GLUE 评估服务器提交了一次结果。<sup>10</sup><https://gluebenchmark.com/leaderboard>

维基百科中包含答案的段落, 任务是预测该段落中答案的文本跨度。

如图1所示, 在问答任务中, 我们将输入的问题和段落表示为单个打包的序列, 其中问题使用 A 嵌入, 段落使用 B 嵌入。我们在微调期间仅引入一个起始向量  $S \in \mathbb{R}^H$  和一个结束向量  $E \in \mathbb{R}^H$ 。词  $i$  作为答案跨度起始位置的概率, 是通过计算  $T_i$  和  $S$  之间的点积, 再对段落中所有词进行 softmax 得到的:

$$P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$$

类似的公式用于答案区间的结尾。从位置  $i$  到位置  $j$  的候选区间的得分定义为  $S \cdot T_i + E \cdot T_j$ , 满足  $j \geq i$  的得分最大的区间被用作预测。训练目标是正确开始位置和结束位置的对数似然之和。我们以5e-5的学习率和32的批次大小进行了3个epoch的微调。

表 2 展示了排行榜前列的条目以及顶级已发表系统的结果 (Seo 等人, 2017; Clark 和 Gardner, 2018; Peters 等人, 2018a; Hu 等人, 2018)。SQuAD 排行榜上的最佳结果没有可用的最新公开系统描述,<sup>11</sup> 并且允许他们在训练系统时使用任何公开数据。因此, 我们在系统中采用了适度的数据增强, 先在 TriviaQA (Joshi 等人, 2017) 上进行微调, 再在 SQuAD 上进行微调。

我们表现最好的系统以集成方式比排行榜上排名最高的系统高出 +1.5 F1, 作为单一系统则高出+1.3 F1。事实上, 我们的单个 BERT 模型在 F1 分数上就超过了排名第一的集成系统。在没有 TriviaQA 微调

<sup>11</sup>QANet 在 Yu 等人(2018)中有所描述, 但该系统自发表后已得到大幅改进。

| System                                   | Dev         |             | Test        |             |
|------------------------------------------|-------------|-------------|-------------|-------------|
|                                          | EM          | F1          | EM          | F1          |
| Top Leaderboard Systems (Dec 10th, 2018) |             |             |             |             |
| Human                                    | -           | -           | 82.3        | 91.2        |
| #1 Ensemble - nlnet                      | -           | -           | 86.0        | 91.7        |
| #2 Ensemble - QANet                      | -           | -           | 84.5        | 90.5        |
| Published                                |             |             |             |             |
| BiDAF+ELMo (Single)                      | -           | 85.6        | -           | 85.8        |
| R.M. Reader (Ensemble)                   | 81.2        | 87.9        | 82.3        | 88.5        |
| Ours                                     |             |             |             |             |
| BERT <sub>BASE</sub> (Single)            | 80.8        | 88.5        | -           | -           |
| BERT <sub>LARGE</sub> (Single)           | 84.1        | 90.9        | -           | -           |
| BERT <sub>LARGE</sub> (Ensemble)         | 85.8        | 91.8        | -           | -           |
| BERT <sub>LARGE</sub> (Sgl.+TriviaQA)    | <b>84.2</b> | <b>91.1</b> | <b>85.1</b> | <b>91.8</b> |
| BERT <sub>LARGE</sub> (Ens.+TriviaQA)    | <b>86.2</b> | <b>92.2</b> | <b>87.4</b> | <b>93.2</b> |

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

| System                                   | Dev  |      | Test |      |
|------------------------------------------|------|------|------|------|
|                                          | EM   | F1   | EM   | F1   |
| Top Leaderboard Systems (Dec 10th, 2018) |      |      |      |      |
| Human                                    | 86.3 | 89.0 | 86.9 | 89.5 |
| #1 Single - MIR-MRC (F-Net)              | -    | -    | 74.8 | 78.0 |
| #2 Single - nlnet                        | -    | -    | 74.2 | 77.1 |
| Published                                |      |      |      |      |
| unet (Ensemble)                          | -    | -    | 71.4 | 74.9 |
| SLQA+ (Single)                           | -    |      | 71.4 | 74.4 |
| Ours                                     |      |      |      |      |
| BERT <sub>LARGE</sub> (Single)           | 78.7 | 81.9 | 80.0 | 83.1 |

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

tuning data, we only lose 0.1-0.4 F1, still outperforming all existing systems by a wide margin.<sup>12</sup>

### 4.3 SQuAD v2.0

The SQuAD 2.0 task extends the SQuAD 1.1 problem definition by allowing for the possibility that no short answer exists in the provided paragraph, making the problem more realistic.

We use a simple approach to extend the SQuAD v1.1 BERT model for this task. We treat questions that do not have an answer as having an answer span with start and end at the [CLS] token. The probability space for the start and end answer span positions is extended to include the position of the [CLS] token. For prediction, we compare the score of the no-answer span:  $s_{\text{null}} = S \cdot C + E \cdot C$  to the score of the best non-null span

<sup>12</sup>The TriviaQA data we used consists of paragraphs from TriviaQA-Wiki formed of the first 400 tokens in documents, that contain at least one of the provided possible answers.

| System                             | Dev         | Test        |
|------------------------------------|-------------|-------------|
| ESIM+GloVe                         | 51.9        | 52.7        |
| ESIM+ELMo                          | 59.1        | 59.2        |
| OpenAI GPT                         | -           | 78.0        |
| BERT <sub>BASE</sub>               | 81.6        | -           |
| BERT <sub>LARGE</sub>              | <b>86.6</b> | <b>86.3</b> |
| Human (expert) <sup>†</sup>        | -           | 85.0        |
| Human (5 annotations) <sup>†</sup> | -           | 88.0        |

Table 4: SWAG Dev and Test accuracies. <sup>†</sup>Human performance is measured with 100 samples, as reported in the SWAG paper.

$s_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$ . We predict a non-null answer when  $s_{i,j} > s_{\text{null}} + \tau$ , where the threshold  $\tau$  is selected on the dev set to maximize F1. We did not use TriviaQA data for this model. We fine-tuned for 2 epochs with a learning rate of 5e-5 and a batch size of 48.

The results compared to prior leaderboard entries and top published work (Sun et al., 2018; Wang et al., 2018b) are shown in Table 3, excluding systems that use BERT as one of their components. We observe a +5.1 F1 improvement over the previous best system.

### 4.4 SWAG

The Situations With Adversarial Generations (SWAG) dataset contains 113k sentence-pair completion examples that evaluate grounded common-sense inference (Zellers et al., 2018). Given a sentence, the task is to choose the most plausible continuation among four choices.

When fine-tuning on the SWAG dataset, we construct four input sequences, each containing the concatenation of the given sentence (sentence A) and a possible continuation (sentence B). The only task-specific parameters introduced is a vector whose dot product with the [CLS] token representation  $C$  denotes a score for each choice which is normalized with a softmax layer.

We fine-tune the model for 3 epochs with a learning rate of 2e-5 and a batch size of 16. Results are presented in Table 4. BERT<sub>LARGE</sub> outperforms the authors’ baseline ESIM+ELMo system by +27.1% and OpenAI GPT by 8.3%.

### 5 Ablation Studies

In this section, we perform ablation experiments over a number of facets of BERT in order to better understand their relative importance. Additional

| 系统                                  | Dev         |             | Test        |             |
|-------------------------------------|-------------|-------------|-------------|-------------|
|                                     | EM          | F1          | EM          | F1          |
| 排行榜前列系统（2018年12月10日）                |             |             |             |             |
| 人类                                  | -           | -           | 82.3        | 91.2        |
| #1 集成系统 - nlnet                     | -           | -           | 86.0        | 91.7        |
| #2 集成 - QANet                       | -           | -           | 84.5        | 90.5        |
| 已发表                                 |             |             |             |             |
| BiDAF+ELMo（单模型）                     | -           | 85.6        | -           | 85.8        |
| R.M. Reader（集成）                     | 81.2        | 87.9        | 82.3        | 88.5        |
| Ours                                |             |             |             |             |
| BERT <sub>BASE</sub> （单模型）          | 80.8        | 88.5        | -           | -           |
| BERT <sub>LARGE</sub> （单模型）         | 84.1        | 90.9        | -           | -           |
| BERT <sub>LARGE</sub> （集成模型）        | 85.8        | 91.8        | -           | -           |
| BERT <sub>LARGE</sub> （单一+TriviaQA） | <b>84.2</b> | <b>91.1</b> | <b>85.1</b> | <b>91.8</b> |
| BERT <sub>LARGE</sub> （集成+TriviaQA） | <b>86.2</b> | <b>92.2</b> | <b>87.4</b> | <b>93.2</b> |

表 2: SQuAD 1.1 结果。BERT 集成模型由 7 个使用不同预训练检查点和微调随机种子的系统组成。

| 系统                          | Dev  |      | Test |      |
|-----------------------------|------|------|------|------|
|                             | EM   | F1   | EM   | F1   |
| 排行榜顶级系统（2018年12月10日）        |      |      |      |      |
| 人类                          | 86.3 | 89.0 | 86.9 | 89.5 |
| #1 单模型 - MIR-MRC (F-Net)    | -    | -    | 74.8 | 78.0 |
| #2 单模型 - nlnet              | -    | -    | 74.2 | 77.1 |
| 发表                          |      |      |      |      |
| unet（集成）                    | -    | -    | 71.4 | 74.9 |
| SLQA+（单模型）                  | -    |      | 71.4 | 74.4 |
| Ours                        |      |      |      |      |
| BERT <sub>LARGE</sub> （单模型） | 78.7 | 81.9 | 80.0 | 83.1 |

表 3: SQuAD 2.0 结果。我们排除了将 BERT 作为其组成部分之一的条目。

数据的情况下，我们仅损失 0.1-0.4 F1，仍以较大优势领先所有现有系统。<sup>12</sup>

#### 4.3 SQuAD v2.0

SQuAD 2.0 任务在 SQuAD 1.1问题定义的基础上进行了扩展，允许所提供的段落中可能不存在简短答案的情况，使问题更加贴近实际。

我们采用一种简单的方法来扩展 SQuAD v1.1 BERT 模型以完成此任务。我们将没有答案的问题视为其答案跨度的起点和终点均在 [CLS] 标记处。答案跨度起始和结束位置的概率空间被扩展为包含 [CLS] 标记的位置。在预测时，我们将无答案跨度的得分： $s_{\text{null}} = S \cdot C + E \cdot C$  与最佳非空跨度的得分进行比较

<sup>12</sup>我们使用的 TriviaQA 数据由 TriviaQA-Wiki 中文档前 400 个 token 构成的段落组成，这些段落至少包含一个所提供的可能答案。

| 系统                    | 开发集         | 测试集         |
|-----------------------|-------------|-------------|
| ESIM+GloVe            | 51.9        | 52.7        |
| ESIM+ELMo             | 59.1        | 59.2        |
| OpenAI GPT            | -           | 78.0        |
| BERT <sub>BASE</sub>  | 81.6        | -           |
| BERT <sub>LARGE</sub> | <b>86.6</b> | <b>86.3</b> |
| 人类（专家） <sup>†</sup>   | -           | 85.0        |
| 人类（5个标注） <sup>†</sup> | -           | 88.0        |

表4: SWAG 开发集和测试集准确率。 <sup>†</sup>人类性能是基于100个样本测量的，如SWAG论文中所述。

$s_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$ . We predict a non-null answer when  $s_{i,j} > s_{\text{null}} + \tau$ , where the threshold  $\tau$  is selected on the dev set to maximize F1. We did not use TriviaQA data for this model. We fine-tuned for 2 epochs with a learning rate of 5e-5 and a batch size of 48.

与先前排行榜条目及已发表顶级工作相比的结果（Sun 等人, 2018; Wang 等人, 2018b) 如表3所示，其中不包括将 BERT 作为其组成部分之一的系统。我们观察到，相较于之前的最佳系统有 +5.1 的 F1 提升。

#### 4.4 SWAG

对抗生成情境（SWAG）数据集包含113k个句子对补全示例，用于评估基于情境的常识推理（Zellers 等人, 2018）。给定一个句子，任务是从四个选项中选择最合理的续写。

在对 SWAG 数据集进行微调时，我们构建了四个输入序列，每个序列包含给定句子(句子A)与一个可能的续写(句子B)的拼接。所引入的唯一任务特定参数是一个向量，该向量与 [CLS] 词元表示的点积  $C$  表示每个选项的得分，并通过 softmax 层进行归一化。

我们以2e-5 的学习率和 16 的批量大小对模型进行了 3 个 epoch 的微调。结果见表 4。BERT<sub>LARGE</sub> 比作者提出的基线 ESIM+ELMo 系统高出 +27.1%，比 OpenAI GPT 高出 8.3%。

### 5 消融研究

在本节中，我们对 BERT 的多个方面进行消融实验，以便更好地理解它们的相对重要性。更多

| Tasks                | Dev Set         |               |               |                |               |
|----------------------|-----------------|---------------|---------------|----------------|---------------|
|                      | MNLI-m<br>(Acc) | QNLI<br>(Acc) | MRPC<br>(Acc) | SST-2<br>(Acc) | SQuAD<br>(F1) |
| BERT <sub>BASE</sub> | 84.4            | 88.4          | 86.7          | 92.7           | 88.5          |
| No NSP               | 83.9            | 84.9          | 86.5          | 92.6           | 87.9          |
| LTR & No NSP         | 82.1            | 84.3          | 77.5          | 92.1           | 77.8          |
| + BiLSTM             | 82.1            | 84.1          | 75.7          | 91.6           | 84.9          |

Table 5: Ablation over the pre-training tasks using the BERT<sub>BASE</sub> architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

ablation studies can be found in Appendix C.

### 5.1 Effect of Pre-training Tasks

We demonstrate the importance of the deep bidirectionality of BERT by evaluating two pre-training objectives using exactly the same pre-training data, fine-tuning scheme, and hyperparameters as BERT<sub>BASE</sub>:

**No NSP:** A bidirectional model which is trained using the “masked LM” (MLM) but without the “next sentence prediction” (NSP) task.

**LTR & No NSP:** A left-context-only model which is trained using a standard Left-to-Right (LTR) LM, rather than an MLM. The left-only constraint was also applied at fine-tuning, because removing it introduced a pre-train/fine-tune mismatch that degraded downstream performance. Additionally, this model was pre-trained without the NSP task. This is directly comparable to OpenAI GPT, but using our larger training dataset, our input representation, and our fine-tuning scheme.

We first examine the impact brought by the NSP task. In Table 5, we show that removing NSP hurts performance significantly on QNLI, MNLI, and SQuAD 1.1. Next, we evaluate the impact of training bidirectional representations by comparing “No NSP” to “LTR & No NSP”. The LTR model performs worse than the MLM model on all tasks, with large drops on MRPC and SQuAD.

For SQuAD it is intuitively clear that a LTR model will perform poorly at token predictions, since the token-level hidden states have no right-side context. In order to make a good faith attempt at strengthening the LTR system, we added a randomly initialized BiLSTM on top. This does significantly improve results on SQuAD, but the

results are still far worse than those of the pre-trained bidirectional models. The BiLSTM hurts performance on the GLUE tasks.

We recognize that it would also be possible to train separate LTR and RTL models and represent each token as the concatenation of the two models, as ELMo does. However: (a) this is twice as expensive as a single bidirectional model; (b) this is non-intuitive for tasks like QA, since the RTL model would not be able to condition the answer on the question; (c) this it is strictly less powerful than a deep bidirectional model, since it can use both left and right context at every layer.

### 5.2 Effect of Model Size

In this section, we explore the effect of model size on fine-tuning task accuracy. We trained a number of BERT models with a differing number of layers, hidden units, and attention heads, while otherwise using the same hyperparameters and training procedure as described previously.

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT<sub>BASE</sub> contains 110M parameters and BERT<sub>LARGE</sub> contains 340M parameters.

It has long been known that increasing the model size will lead to continual improvements on large-scale tasks such as machine translation and language modeling, which is demonstrated by the LM perplexity of held-out training data shown in Table 6. However, we believe that this is the first work to demonstrate convincingly that scaling to extreme model sizes also leads to large improvements on very small scale tasks, provided that the model has been sufficiently pre-trained. Peters et al. (2018b) presented

| 任务                   | 开发集             |               |               |                |               |
|----------------------|-----------------|---------------|---------------|----------------|---------------|
|                      | MNLI-m<br>(准确率) | QNLI<br>(准确率) | MRPC<br>(准确率) | SST-2<br>(准确率) | SQuAD<br>(F1) |
| BERT <sub>BASE</sub> | 84.4            | 88.4          | 86.7          | 92.7           | 88.5          |
| 无 NSP                | 83.9            | 84.9          | 86.5          | 92.6           | 87.9          |
| LTR & 无 NSP          | 82.1            | 84.3          | 77.5          | 92.1           | 77.8          |
| + BiLSTM             | 82.1            | 84.1          | 75.7          | 91.6           | 84.9          |

表 5: 使用 BERT<sub>BASE</sub> 架构对预训练任务进行的消融研究。“无 NSP”是在没有下一句预测任务的情况下训练的。“LTR & 无 NSP”是在没有下一句预测的情况下作为从左到右的语言模型进行训练的，类似于 OpenAI GPT。“+ BiLSTM”是在微调期间在“LTR + 无 NSP”模型之上添加了一个随机初始化的 BiLSTM。

消融研究可参见附录C。

### 5.1 预训练任务的影响

我们通过使用与BERT<sub>BASE</sub>完全相同的预训练数据、微调方案和超参数来评估两种预训练目标，从而证明了BERT深度双向性的重要性：

**无NSP:** 一种使用“掩码语言模型” (MLM) 但不包含“下一句预测” (NSP) 任务训练的双向模型。

**LTR且无NSP:** 一种仅使用左上下文的模型，使用标准的从左到右 (LTR) 语言模型而非 MLM进行训练。仅左约束在微调阶段也被应用，因为移除它会引入预训练/微调不匹配，从而降低下游性能。此外，该模型在预训练时未使用NSP任务。这可以直接与OpenAI GPT进行比较，但使用了我们更大的训练数据集、我们的输入表示和我们的微调方案。

我们首先检验了NSP任务带来的影响。在表 5中，我们展示了移除NSP在QNLI、MNLI和SQuAD 1.1上显著损害了性能。接下来，我们通过比较“无NSP”与“LTR且无NSP”来评估训练双向表示的影响。LTR模型在所有任务上的表现均差于MLM模型，在MRPC和SQuAD上下降幅度较大。

对于 SQuAD 而言，LTR 模型在词元预测上表现不佳是显而易见的，因为词元级别的隐藏状态没有右侧上下文。为了切实尝试强化 LTR 系统，我们在其上添加了一个随机初始化的 BiLSTM。这确实显著改善了 SQuAD 上的结果，但

结果仍远逊于预训练的双向模型。BiLSTM会损害 GLUE 任务上的性能。

我们认识到，也可以像 ELMo 那样分别训练从左到右 (LTR) 和从右到左 (RTL) 模型，并将每个 token 表示为两个模型输出的拼接。然而：(a) 这种做法的开销是单个双向模型的两倍；(b) 对于问答 (QA) 这类任务而言不够直观，因为 RTL 模型无法基于问题来生成答案；(c) 这种方法的表达能力严格弱于深度双向模型，因为深度双向模型在每一层都能同时利用左右两侧的上文。

### 5.2 模型规模的影响

在本节中，我们探讨了模型规模对微调任务准确率的影响。我们训练了多个 BERT 模型，这些模型的层数、隐藏单元数和注意力头数各不相同，其余方面则采用与前文所述相同的超参数和训练流程。

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT<sub>BASE</sub> contains 110M parameters and BERT<sub>LARGE</sub> contains 340M parameters.

人们早已知道，增大模型规模会在机器翻译和语言建模等大规模任务上带来持续的提升，这一点已由表6中留出的训练数据上的语言模型困惑度所证明。然而，我们认为，这是首个令人信服地证明只要模型经过了充分的预训练，将模型扩展到极端规模也能在非常小规模的任务上带来巨大提升的工作。Peters等人 (2018b)给出了

mixed results on the downstream task impact of increasing the pre-trained bi-LM size from two to four layers and Melamud et al. (2016) mentioned in passing that increasing hidden dimension size from 200 to 600 helped, but increasing further to 1,000 did not bring further improvements. Both of these prior works used a feature-based approach — we hypothesize that when the model is fine-tuned directly on the downstream tasks and uses only a very small number of randomly initialized additional parameters, the task-specific models can benefit from the larger, more expressive pre-trained representations even when downstream task data is very small.

### 5.3 Feature-based Approach with BERT

All of the BERT results presented so far have used the fine-tuning approach, where a simple classification layer is added to the pre-trained model, and all parameters are jointly fine-tuned on a downstream task. However, the feature-based approach, where fixed features are extracted from the pre-trained model, has certain advantages. First, not all tasks can be easily represented by a Transformer encoder architecture, and therefore require a task-specific model architecture to be added. Second, there are major computational benefits to pre-compute an expensive representation of the training data once and then run many experiments with cheaper models on top of this representation.

In this section, we compare the two approaches by applying BERT to the CoNLL-2003 Named Entity Recognition (NER) task (Tjong Kim Sang and De Meulder, 2003). In the input to BERT, we use a case-preserving WordPiece model, and we include the maximal document context provided by the data. Following standard practice, we formulate this as a tagging task but do not use a CRF

| Hyperparams |      |    |          | Dev Set Accuracy |      |       |  |
|-------------|------|----|----------|------------------|------|-------|--|
| #L          | #H   | #A | LM (ppl) | MNLI-m           | MRPC | SST-2 |  |
| 3           | 768  | 12 | 5.84     | 77.9             | 79.8 | 88.4  |  |
| 6           | 768  | 3  | 5.24     | 80.6             | 82.2 | 90.7  |  |
| 6           | 768  | 12 | 4.68     | 81.9             | 84.8 | 91.3  |  |
| 12          | 768  | 12 | 3.99     | 84.4             | 86.7 | 92.9  |  |
| 12          | 1024 | 16 | 3.54     | 85.7             | 86.9 | 93.3  |  |
| 24          | 1024 | 16 | 3.23     | 86.6             | 87.8 | 93.7  |  |

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. “LM (ppl)” is the masked LM perplexity of held-out training data.

| System                                         | Dev F1 | Test F1     |
|------------------------------------------------|--------|-------------|
| ELMo (Peters et al., 2018a)                    | 95.7   | 92.2        |
| CVT (Clark et al., 2018)                       | -      | 92.6        |
| CSE (Akbik et al., 2018)                       | -      | <b>93.1</b> |
| Fine-tuning approach                           |        |             |
| BERT <sub>LARGE</sub>                          | 96.6   | 92.8        |
| BERT <sub>BASE</sub>                           | 96.4   | 92.4        |
| Feature-based approach (BERT <sub>BASE</sub> ) |        |             |
| Embeddings                                     | 91.0   | -           |
| Second-to-Last Hidden                          | 95.6   | -           |
| Last Hidden                                    | 94.9   | -           |
| Weighted Sum Last Four Hidden                  | 95.9   | -           |
| Concat Last Four Hidden                        | 96.1   | -           |
| Weighted Sum All 12 Layers                     | 95.5   | -           |

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

layer in the output. We use the representation of the first sub-token as the input to the token-level classifier over the NER label set.

To ablate the fine-tuning approach, we apply the feature-based approach by extracting the activations from one or more layers *without* fine-tuning any parameters of BERT. These contextual embeddings are used as input to a randomly initialized two-layer 768-dimensional BiLSTM before the classification layer.

Results are presented in Table 7. BERT<sub>LARGE</sub> performs competitively with state-of-the-art methods. The best performing method concatenates the token representations from the top four hidden layers of the pre-trained Transformer, which is only 0.3 F1 behind fine-tuning the entire model. This demonstrates that BERT is effective for both fine-tuning and feature-based approaches.

## 6 Conclusion

Recent empirical improvements due to transfer learning with language models have demonstrated that rich, unsupervised pre-training is an integral part of many language understanding systems. In particular, these results enable even low-resource tasks to benefit from deep unidirectional architectures. Our major contribution is further generalizing these findings to deep *bidirectional* architectures, allowing the same pre-trained model to successfully tackle a broad set of NLP tasks.

将预训练bi-LM的规模从两层增加到四层对下游任务的影响结果好坏参半，并且 Melamud等人 (2016) 顺带提到，将隐藏维度从200增加到600有所帮助，但进一步增加到1,000并未带来进一步的改进。这两项先前的工作都采用了基于特征的方法——我们假设，当模型直接在下游任务上进行微调，并且仅使用数量极少的随机初始化附加参数时，即使下游任务数据非常少，任务特定的模型也能从更大、更具表现力的预训练表示中受益。

### 5.3 基于 BERT 的特征方法

迄今为止介绍的所有 BERT 结果均采用了微调方法，即在预训练模型上添加一个简单的分类层，并在下游任务上对所有参数进行联合微调。然而，基于特征的方法——即从预训练模型中提取固定特征——也具有某些优势。首先，并非所有任务都能轻松地用 Transformer 编码器架构来表示，因此需要添加针对特定任务的模型架构。其次，预先对训练数据一次性计算其高成本的表示，然后在此表示之上用更廉价的模型运行大量实验，可以带来显著的计算优势。

在本节中，我们通过将 BERT 应用于 CoNLL-2003 命名实体识别（NER）任务来比较这两种方法（Tjong Kim Sang and De Meulder, 2003）。在 BERT 的输入中，我们使用保留大小写的 WordPiece 模型，并包含数据所提供的最大文档上下文。按照标准做法，我们将其表述为一个标注任务，但不使用 CRF

| 超参数 |      |    |          | 开发集准确率 |      |       |  |
|-----|------|----|----------|--------|------|-------|--|
| #L  | #H   | #A | LM (ppl) | MNLI-m | MRPC | SST-2 |  |
| 3   | 768  | 12 | 5.84     | 77.9   | 79.8 | 88.4  |  |
| 6   | 768  | 3  | 5.24     | 80.6   | 82.2 | 90.7  |  |
| 6   | 768  | 12 | 4.68     | 81.9   | 84.8 | 91.3  |  |
| 12  | 768  | 12 | 3.99     | 84.4   | 86.7 | 92.9  |  |
| 12  | 1024 | 16 | 3.54     | 85.7   | 86.9 | 93.3  |  |
| 24  | 1024 | 16 | 3.23     | 86.6   | 87.8 | 93.7  |  |

表6：BERT模型大小的消融实验。#L = 层数；#H =隐藏层大小；#A = 注意力头数量。“LM (ppl)” 是留出训练数据的掩码LM困惑度。

| 系统                              | 开发集 F1 | 测试集 F1      |
|---------------------------------|--------|-------------|
| ELMo (Peters等人, 2018a)          | 95.7   | 92.2        |
| CVT (Clark等人, 2018)             | -      | 92.6        |
| CSE (Akbik等人, 2018)             | -      | <b>93.1</b> |
| 微调方法                            |        |             |
| BERT <sub>LARGE</sub>           | 96.6   | 92.8        |
| BERT <sub>BASE</sub>            | 96.4   | 92.4        |
| 基于特征的方法 (BERT <sub>BASE</sub> ) |        |             |
| 嵌入                              | 91.0   | -           |
| 倒数第二层隐藏层                        | 95.6   | -           |
| 最后一层隐藏层                         | 94.9   | -           |
| 最后四层隐藏层加权                       | 95.9   | -           |
| 拼接最后四层隐藏层                       | 96.1   | -           |
| 对全部 12 层加权求和                    | 95.5   | -           |

表 7: CoNLL-2003 命名实体识别结果。超参数使用开发集选取。所报告的开发集和测试集得分为使用这些超参数进行 5 次随机重启后的平均值。

层作为输出。我们使用第一个子词的表示作为 NER 标签集上词元级分类器的输入。

为了消融微调方法，我们采用基于特征的方法，从一层或多层中提取激活值而无需微调 BERT 的任何参数。这些上下文嵌入被用作分类层之前随机初始化的两层 768 维 BiLSTM 的输入。

结果见表7。BERT<sub>LARGE</sub>与最先进方法相比具有竞争力。表现最佳的方法拼接了预训练 Transformer 顶部四个隐藏层的词元表示，仅比微调整个模型落后 0.3 F1。这表明 BERT 对微调和基于特征两种方法都有效。

## 6 结论

近来，借助语言模型进行迁移学习所取得的实证改进表明，丰富的无监督预训练是许多语言理解系统不可或缺的组成部分。特别是，这些结果使得即便是低资源任务也能从深度单向架构中受益。我们的主要贡献是将这些发现进一步推广到深度双向 架构，使同一个预训练模型能够成功应对广泛的NLP任务。

## References

Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.

Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.

Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC*. NIST.

John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.

Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. **Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation**. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.

Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.

Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. **Quora question pairs**.

Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.

Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.

Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.

Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. **Supervised learning of universal sentence representations from natural language inference data**. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.

Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.

William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.

William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the... *arXiv preprint arXiv:1801.07736*.

Dan Hendrycks and Kevin Gimpel. 2016. **Bridging nonlinearities and stochastic regularizers with gaussian error linear units**. *CoRR*, abs/1606.08415.

Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.

Jeremy Howard and Sebastian Ruder. 2018. **Universal language model fine-tuning for text classification**. In *ACL*. Association for Computational Linguistics.

Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.

Yacine Jernite, Samuel R. Bowman, and David Sontag. 2017. **Discourse-based objectives for fast unsupervised sentence representation learning**. *CoRR*, abs/1705.00557.

## 参考文献

Alan Akbik、Duncan Blythe 和 Roland Vollgraf. 2018. 用于序列标注的上下文字符串嵌入。载于第27届国际计算语言学会会议论文集，第1638–1649页。

Rami Al-Rfou、Dokook Choe、Noah Constant、Mandy Guo 和 Llion Jones，2018年。基于更深自注意力机制的字符级语言建模。*arXiv* 预印本 *arXiv:1808.04444*。

Rie Kubota Ando 和 Tong Zhang，2005年。一种从多任务和无标注数据中学习预测结构的框架。机器学习研究杂志，6(Nov):1817–1853。

Luisa Bentivogli、Bernardo Magnini、Ido Dagan、Hoa Trang Dang 和 Danilo Giampiccolo，2009年。第五届 PASCAL 文本蕴涵识别挑战赛。见TAC。NIST。

John Blitzer、Ryan McDonald 和 Fernando Pereira，2006年。基于结构对应学习的领域自适应。见2006年自然语言处理经验方法会议论文集，第120–128页。计算语言学协会。

Samuel R. Bowman, Gabor Angeli, Christopher Potts和 Christopher D. Manning. 2015. 用于学习自然语言推理的大型标注语料库.收录于 *EMNLP*. 计算语言学协会. Peter F Brown, Peter V Desouza, Robert L Mercer,Vincent J Della Pietra 和 Jenifer C Lai. 1992.基于类别的自然语言 n-gram 模型。计算语言学, 18(4):467–479.Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio 和 Lucia Specia. 2017. Semeval-2017任务 1:语义文本相似度多语言与跨语言专项评估. 收录于 第11届语义评估国际研讨会(SemEval-2017)论文集, 第 1–14 页, 加拿大温哥华 . 计算语言学协会 . Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge,Thorsten Brants, Phillipp Koehn 和 Tony Robin-son. 2013. 用于衡量统计语言建模进展的十亿词基准. arXiv预印本 arXiv:1312.3005.Z. Chen, H. Zhang, X. Zhang 和 L. Zhao. 2018.Quora 问题对. Christopher Clark 和 Matt Gardner. 2018. 简单有效的多段落阅读理解. 收录于ACL.

Kevin Clark、Minh-Thang Luong、Christopher D Manning 和 Quoc Le. 2018. 基于交叉视图训练的半监督序列建模。载于 2018年自然语言处理实证方法会议论文集，第1914–1925页。

Ronan Collobert 和 Jason Weston. 2008年。自然语言处理的统一架构：基于多任务学习的深度神经网络。载于第25届国际机器学习会议论文集，第160–167页。ACM。

Alexis Conneau、Douwe Kiela、Holger Schwenk、Loïc Barrault 和 Antoine Bordes, 2017年。**基于自然语言推理数据的通用句子表示监督学习**。收录于2017年自然语言处理实证方法会议论文集，第670–680页，丹麦哥本哈根。计算语言学协会。

Andrew M Dai 和 Quoc V Le. 2015. 半监督序列学习. 载于 神经信息处理系统进展, 第3079–3087页。

J. Deng、W. Dong、R. Socher、L.-J. Li、K. Li 和 L. Fei-Fei. 2009年。ImageNet：一个大规模分层图像数据库。发表于*CVPR09*。

William B Dolan 和 Chris Brockett. 2005年。自动构建句子级复述语料库。收录于 第三届国际复述研讨会 (*IWP2005*) 论文集. William Fedus、Ian Goodfellow 和 Andrew M Dai. 2018年.Maskgan：通过填空实现更好的文本生成。 *arXiv*预印本 *arXiv:1801.07736*. Dan Hendrycks 和 Kevin Gimpel. 2016年。 **用高斯误差线性单元桥接非线性与随机正则化器**. *CoRR*, abs/1606.08415. Felix Hill、Kyunghyun Cho 和 Anna Korhonen. 2016年。从无标注数据中学习句子的分布式表示。收录于 2016年计算语言学协会北美分会会议论文集：人类语言技术。计算语言学协会. Jeremy Howard 和 Sebastian Ruder. 2018年。 **面向文本分类的通用语言模型微调**. 收录于ACL. 计算语言学协会. Minghao Hu、Yuxing Peng、Zhen Huang、Xipeng Qiu、Furu Wei 和 Ming Zhou. 2018年。用于机器阅读理解的强化记忆阅读器. 收录于*IJCAI*. Yacine Jernite、Samuel R. Bowman 和 David Sontag. 2017年。 **基于篇章的快速无监督句子表示学习目标**. *CoRR*, abs/1705.00557。

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.

Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.

Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.

Lajanugen Logeswaran and Honglak Lee. 2018. [An efficient framework for learning sentence representations](#). In *International Conference on Learning Representations*.

Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.

Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.

Andriy Mnih and Geoffrey E Hinton. 2009. [A scalable hierarchical distributed language model](#). In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.

Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [Glove: Global vectors for word representation](#). In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Matthew Peters, Waleed Ammar, Chandra Bhagavatula, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.

Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.

Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.

Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment tree-bank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.

Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.

Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.

Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.

Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL ’10*, pages 384–394.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform

Mandar Joshi, Eunsol Choi, Daniel S Weld 和 Luke Zettlemoyer. 2017. TriviaQA: 一个面向阅读理解的大规模远程监督挑战数据集。发表于*ACL*。

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba和 Sanja Fidler. 2015. Skip-thought向量。载于神经信息处理系统进展，第3294–3302页。

Quoc Le 和 Tomas Mikolov. 2014. 句子与文档的分布式表示。载于 国际机器学习会议，第1188–1196页。

Hector J Levesque, Ernest Davis 和 Leora Morgenstern. 2011. Winograd模式挑战。载于AAAI春季研讨会：常识推理的逻辑形式化，第46卷，第47页。

Lajanugen Logeswaran 和 Honglak Lee. 2018. [学习句子表示的高效框架](#)。载于 国际学习表征会议。

Bryan McCann, James Bradbury, Caiming Xiong 和 Richard Socher. 2017. 从翻译中学习：语境化词向量。载于*NIPS*。

Oren Melamud, Jacob Goldberger 和 Ido Dagan. 2016年. context2vec: 利用双向 LSTM 学习通用上下文嵌入。收录于*CoNLL*。Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado 和 Jeff Dean. 2013年. 词与短语的分布式表示及其组合性。收录于 神经信息处理系统进展 26，第 3111–3119 页。Curran Associates, Inc.Andriy Mnih 和 Geoffrey E Hinton. 2009年.

[一种可扩展的层级分布式语言模型](#)。收录于D. Koller, D. Schuurmans, Y. Bengio 和 L. Bottou 编，神经信息处理系统进展 21，第 1081–1088 页。Curran Associates, Inc.Ankur P Parikh, Oscar Täckström, Dipanjan Das 和 Jakob Uszkoreit. 2016年. 一种用于自然语言推理的可分解注意力模型。收录于*EMNLP*。Jeffrey Pennington, Richard Socher 和 Christopher D. Manning. 2014年. [Glove: 词表示的全局向量](#)。收录于 自然语言处理经验方法 (*EMNLP*)，第 1532–1543 页。Matthew Peters, Waleed Ammar, Chandra Bhagavatula 和 Russell Power. 2017年. 基于双向语言模型的半监督序列标注。收录于 *ACL*。

Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee 和 Luke Zettlemoyer. 2018a. 深度语境化词表示。收录于*NAACL*。

Matthew Peters, Mark Neumann, Luke Zettlemoyer 和 Wen-tau Yih. 2018b. 剖析语境化词嵌入：架构与表示。收录于*2018年自然语言处理实证方法会议论文集*，第1499–1509页。

Alec Radford, Karthik Narasimhan, Tim Salimans 和 Ilya Sutskever. 2018年. 利用无监督学习改进语言理解。技术报告，OpenAI。

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev 和 Percy Liang. 2016年. Squad: 100,000+ 个面向文本机器理解的问题。发表于 *2016年自然语言处理实证方法会议论文集*，第2383–2392页。

Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi 和 Hannaneh Hajishirzi. 2017年. 面向机器理解的双向注意力流。发表于*ICLR*。

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng 和 Christopher Potts. 2013年. 基于情感树库的语义组合性递归深度模型。发表于 *2013年自然语言处理实证方法会议论文集*，第1631–1642页。

Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.

Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.

Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.

Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL ’10*, pages 384–394.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform

for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

## Appendix for “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

We organize the appendix into three sections:

- Additional implementation details for BERT are presented in Appendix A;

- Additional details for our experiments are presented in Appendix B; and

- Additional ablation studies are presented in Appendix C.

We present additional ablation studies for BERT including:

- Effect of Number of Training Steps; and
- Ablation for Different Masking Procedures.

## A Additional Details for BERT

### A.1 Illustration of the Pre-training Tasks

We provide examples of the pre-training tasks in the following.

**Masked LM and the Masking Procedure** Assuming the unlabeled sentence is `my dog is hairy`, and during the random masking procedure we chose the 4-th token (which corresponding to `hairy`), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the [MASK] token, e.g., `my dog is hairy` → `my dog is [MASK]`

- 10% of the time: Replace the word with a random word, e.g., `my dog is hairy` → `my dog is apple`

- 10% of the time: Keep the word unchanged, e.g., `my dog is hairy` → `my dog is hairy`. The purpose of this is to bias the representation towards the actual observed word.

The advantage of this procedure is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words, so it is forced to keep a distributional contextual representation of every input token. Additionally, because random replacement only occurs for 1.5% of all tokens (i.e., 10% of 15%), this does not seem to harm the model’s language understanding capability. In Section C.2, we evaluate the impact this procedure.

Compared to standard language model training, the masked LM only make predictions on 15% of tokens in each batch, which suggests that more pre-training steps may be required for the model

面向自然语言理解。载于2018年EMNLP研讨会*BlackboxNLP: 自然语言处理神经网络分析与解读论文集*，第353–355页。

Wei Wang、Ming Yan 和 Chen Wu。2018b。《面向阅读理解与问答的多粒度层次注意力融合网络》。收录于第56届计算语言学协会年会论文集（第1卷：长文）。计算语言学协会。

Alex Warstadt、Amanpreet Singh 和 Samuel R Bowman。2018。《神经网络可接受性判断》。*arXiv* 预印本 *arXiv:1805.12471*。

Adina Williams、Nikita Nangia 和 Samuel R Bowman。2018。《通过推理实现句子理解的广覆盖挑战语料库》。收录于NAACL。

Yonghui Wu、Mike Schuster、Zhifeng Chen、Quoc V Le、Mohammad Norouzi、Wolfgang Macherey、Maxim Krikun、Yuan Cao、Qin Gao、Klaus Macherey 等。2016。《谷歌神经机器翻译系统：弥合人类翻译与机器翻译之间的差距》。*arXiv* 预印本 *arXiv:1609.08144*。

Jason Yosinski、Jeff Clune、Yoshua Bengio 和 Hod Lipson。2014年。深度神经网络中的特征可迁移性如何？发表于神经信息处理系统进展，第3320–3328页。

Adams Wei Yu、David Dohan、Minh-Thang Luong、Rui Zhao、Kai Chen、Mohammad Norouzi 和 Quoc V Le。2018年。QANet：结合局部卷积与全局自注意力机制进行阅读理解。发表于ICLR。

Rowan Zellers、Yonatan Bisk、Roy Schwartz 和 Yejin Choi。2018年。Swag：一个用于情境化常识推理的大规模对抗性数据集。发表于2018年自然语言处理实证方法会议（EMNLP）论文集。

Yukun Zhu、Ryan Kiros、Rich Zemel、Ruslan Salakhutdinov、Raquel Urtasun、Antonio Torralba 和 Sanja Fidler。2015年。对齐书籍与电影：通过观看电影和阅读书籍获得故事般的视觉解释。发表于IEEE国际计算机视觉会议论文集，第19–27页。

## 《BERT：用于语言理解的深度双向Transformer 预训练》附录

我们将附录组织为三个部分：

- BERT 的其他实现细节见附录A；

- 我们实验的其他细节见附录 B；以及

- 额外的消融研究见附录 C。

我们为BERT 提供了额外的消融研究，包括：

- 训练步数的影响；以及– 不同掩码程序的消融实验。

## A BERT 的其他细节

### A.1 预训练任务示例

我们在下文中提供预训练任务的示例。

**掩码语言模型与掩码过程** 假设未标注的句子是 `my dogishairy`，并且在随机掩码过程中我们选择了第4个词元（对应`hairy`），我们的掩码过程可以通过如下方式进一步说明

- 80% 的情况下：将单词替换为[MASK]标记，例如：`my dog is hairy` → `my dog is [MASK]`

- 10%的时间：将该单词替换为一个随机词，例如，`我的狗毛发浓密` → `我的狗是苹果`

- 10% 的情况下：保持该词不变，例如，`我的狗毛很多` → `我的狗毛很多`。这样做的目的是使表示偏向于实际观察到的词。

此方法的优点在于，Transformer 编码器并不知道它将被要求预测哪些词，或者哪些词已被随机词替换，因此它被迫为每个输入词元保留分布式的上下文表示。此外，由于随机替换仅发生于全部词元的 1.5%（即 15% 的 10%），这似乎并不会损害模型的语言理解能力。在第 C.2 节中，我们评估了此方法的影响。

与标准的语言模型训练相比，掩码语言模型（MLM）仅对每个批次中15%的词符进行预测，这表明该模型可能需要更多的预训练

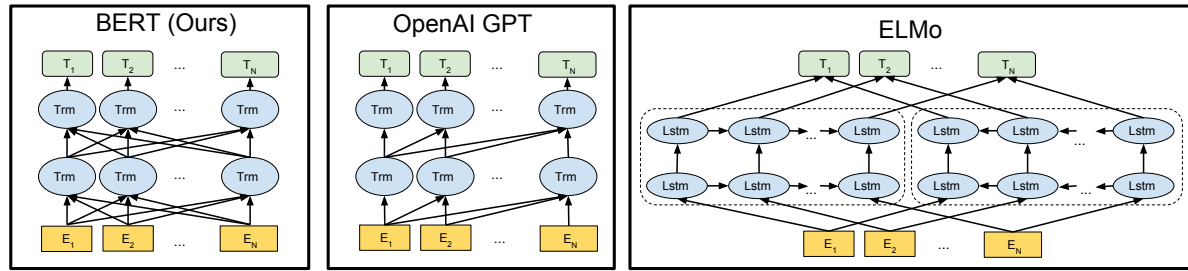


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

to converge. In Section C.1 we demonstrate that MLM does converge marginally slower than a left-to-right model (which predicts every token), but the empirical improvements of the MLM model far outweigh the increased training cost.

**Next Sentence Prediction** The next sentence prediction task can be illustrated in the following examples.

Input = [CLS] the man went to [MASK] store [SEP]  
he bought a gallon [MASK] milk [SEP]  
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]  
penguin [MASK] are flight ##less birds [SEP]  
Label = NotNext

## A.2 Pre-training Procedure

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as “sentences” even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the A embedding and the second receives the B embedding. 50% of the time B is the actual next sentence that follows A and 50% of the time it is a random sentence, which is done for the “next sentence prediction” task. They are sampled such that the combined length is  $\leq 512$  tokens. The LM masking is applied after WordPiece tokenization with a uniform masking rate of 15%, and no special consideration given to partial word pieces.

We train with batch size of 256 sequences (256 sequences \* 512 tokens = 128,000 tokens/batch) for 1,000,000 steps, which is approximately 40

epochs over the 3.3 billion word corpus. We use Adam with learning rate of  $1e-4$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a gelu activation (Hendrycks and Gimpel, 2016) rather than the standard relu, following OpenAI GPT. The training loss is the sum of the mean masked LM likelihood and the mean next sentence prediction likelihood.

Training of BERT<sub>BASE</sub> was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).<sup>13</sup> Training of BERT<sub>LARGE</sub> was performed on 16 Cloud TPUs (64 TPU chips total). Each pre-training took 4 days to complete.

Longer sequences are disproportionately expensive because attention is quadratic to the sequence length. To speed up pretraining in our experiments, we pre-train the model with sequence length of 128 for 90% of the steps. Then, we train the rest 10% of the steps of sequence of 512 to learn the positional embeddings.

## A.3 Fine-tuning Procedure

For fine-tuning, most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. The dropout probability was always kept at 0.1. The optimal hyperparameter values are task-specific, but we found the following range of possible values to work well across all tasks:

- **Batch size:** 16, 32

<sup>13</sup><https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

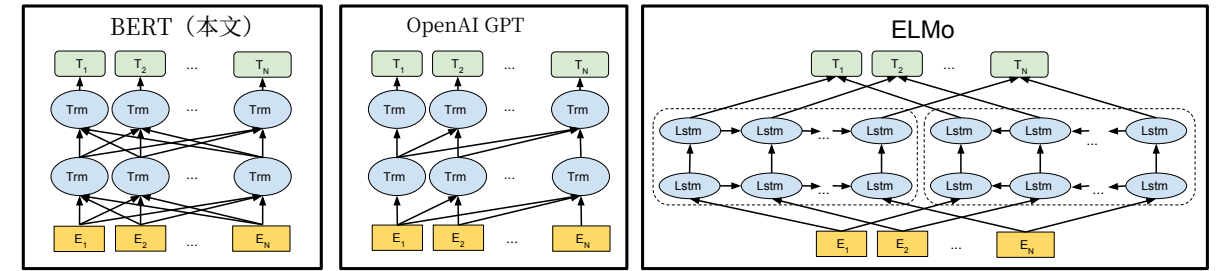


图 3：预训练模型架构的差异。BERT 使用双向 Transformer。OpenAI GPT 使用从左到右的 Transformer。ELMo 使用独立训练的从左到右与从右到左 LSTMs 的拼接来为下游任务生成特征。在三者之中，只有 BERT 的表示在所有层中都同时依赖于左右两侧的上下文。除了架构差异之外，BERT 和 OpenAI GPT 属于微调方法，而 ELMo 属于基于特征的方法。

步骤才能收敛。在第 C.1 节中，我们证明了 MLM 的收敛速度仅略慢于从左到右模型（该模型预测每个词符），但 MLM 模型的实证改进远远超过了所增加的训练成本。

**下一句预测** 下一句预测任务可以通过以下示例来说明。

输入 = [CLS] 那个男人去了 [MASK] 商店 [SEP] 他买了一加仑 [MASK] 牛奶 [SEP] 标签 = IsNext  
输入 = [CLS] 那个男人 [MASK] 商店 [SEP] 企鹅 [MASK] 是不会飞的鸟 [SEP] 标签 = NotNext

## A.2 预训练过程

为了生成每个训练输入序列，我们从语料库中采样两段文本，并将其称为“句子”，尽管它们通常比单个句子长得多（但也可能更短）。第一个句子被赋予 A 嵌入，第二个句子被赋予 B 嵌入。50% 的情况下，B 是紧跟 A 之后的实际下一句；另外 50% 的情况下，B 是随机选取的句子，这样做是为了完成“下一句预测”任务。采样时使两者的总长度为  $\leq 512$  个词元。语言模型（LM）掩码在 WordPiece 分词之后应用，采用 15% 的统一掩码率，且未对不完整的词片作特殊处理。

我们以 256 个序列的批次大小进行训练（256 个序列 \* 512 个 token = 128,000 个 token/批次）共训练 1,000,000 步，这大约是 40

个轮次遍历 33 亿单词的语料库。我们使用 Adam 优化器，学习率为  $1e-4$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , L2 权重衰减系数为 0.01，在前 10,000 步内进行学习率预热，并对学习率进行线性衰减。我们在所有层上使用 0.1 的 dropout 概率。我们使用 gelu 激活函数 (Hendrycks 和 Gimpel, 2016) 而非标准的 relu，遵循 OpenAI GPT 的做法。训练损失为掩码语言模型平均似然与下一句预测平均似然之和。

BERT<sub>BASE</sub> 的训练是在 4 个采用 Pod 配置的 Cloud TPU（共 16 个 TPU 芯片）上进行的。<sup>13</sup> BERT<sub>LARGE</sub> 的训练是在 16 个 Cloud TPU（共 64 个 TPU 芯片）上进行的。每次预训练耗时 4 天完成。

较长的序列代价尤其高昂，因为注意力机制的计算量与序列长度呈平方关系。为了在我们的实验中加快预训练速度，我们在 90% 的训练步骤中使用 128 的序列长度对模型进行预训练。然后，在剩余 10% 的步骤中使用 512 的序列长度进行训练，以学习位置嵌入。

## A.3 微调过程

在微调时，除批量大小、学习率和训练轮数之外，大多数模型超参数与预训练时相同。Dropout 概率始终保持为 0.1。最优超参数值因任务而异，但我们发现以下取值范围在所有任务上均能良好地工作：

- **批量大小:** 16, 32

<sup>13</sup><https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

- **Learning rate (Adam):** 5e-5, 3e-5, 2e-5
- **Number of epochs:** 2, 3, 4

We also observed that large data sets (e.g., 100k+ labeled training examples) were far less sensitive to hyperparameter choice than small data sets. Fine-tuning is typically very fast, so it is reasonable to simply run an exhaustive search over the above parameters and choose the model that performs best on the development set.

#### A.4 Comparison of BERT, ELMo ,and OpenAI GPT

Here we studies the differences in recent popular representation learning models including ELMo, OpenAI GPT and BERT. The comparisons between the model architectures are shown visually in Figure 3. Note that in addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

The most comparable existing pre-training method to BERT is OpenAI GPT, which trains a left-to-right Transformer LM on a large text corpus. In fact, many of the design decisions in BERT were intentionally made to make it as close to GPT as possible so that the two methods could be minimally compared. The core argument of this work is that the bi-directionality and the two pre-training tasks presented in Section 3.1 account for the majority of the empirical improvements, but we do note that there are several other differences between how BERT and GPT were trained:

- GPT is trained on the BooksCorpus (800M words); BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words).
- GPT uses a sentence separator ([SEP]) and classifier token ([CLS]) which are only introduced at fine-tuning time; BERT learns [SEP], [CLS] and sentence A/B embeddings during pre-training.
- GPT was trained for 1M steps with a batch size of 32,000 words; BERT was trained for 1M steps with a batch size of 128,000 words.
- GPT used the same learning rate of 5e-5 for all fine-tuning experiments; BERT chooses a task-specific fine-tuning learning rate which performs the best on the development set.

To isolate the effect of these differences, we perform ablation experiments in Section 5.1 which demonstrate that the majority of the improvements are in fact coming from the two pre-training tasks and the bidirectionality they enable.

#### A.5 Illustrations of Fine-tuning on Different Tasks

The illustration of fine-tuning BERT on different tasks can be seen in Figure 4. Our task-specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure,  $E$  represents the input embedding,  $T_i$  represents the contextual representation of token  $i$ , [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

### B Detailed Experimental Setup

#### B.1 Detailed Descriptions for the GLUE Benchmark Experiments.

Our GLUE results in Table1 are obtained from <https://gluebenchmark.com/leaderboard> and <https://blog.openai.com/language-unsupervised>. The GLUE benchmark includes the following datasets, the descriptions of which were originally summarized in Wang et al. (2018a):

**MNLI** Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an *entailment*, *contradiction*, or *neutral* with respect to the first one.

**QQP** Quora Question Pairs is a binary classification task where the goal is to determine if two questions asked on Quora are semantically equivalent (Chen et al., 2018).

**QNLI** Question Natural Language Inference is a version of the Stanford Question Answering Dataset (Rajpurkar et al., 2016) which has been converted to a binary classification task (Wang et al., 2018a). The positive examples are (question, sentence) pairs which do contain the correct answer, and the negative examples are (question, sentence) from the same paragraph which do not contain the answer.

- **学习率 (Adam)** : 5e-5, 3e-5, 2e-5
- **训练轮数:** 2, 3, 4

我们还观察到, 与小型数据集相比, 大型数据集 (例如 100k+ 个标注训练样本) 对超参数选择的敏感度要低得多。微调通常非常快, 因此直接对上述参数进行穷举搜索, 并选择在开发集上表现最佳的模型, 是合理的做法。

#### A.4 BERT、ELMo 与 OpenAI GPT 的比较

在本节中, 我们研究了近期流行的几种表示学习模型 (包括 ELMo、OpenAI GPT 和 BERT) 之间的差异。模型架构之间的比较以可视化形式展示于图3中。请注意, 除了架构上的差异之外, BERT 和 OpenAI GPT 属于微调方法, 而 ELMo 则是一种基于特征的方法。

与 BERT 最具可比性的现有预训练方法是 OpenAI GPT, 它在大规模文本语料库上训练一个从左到右的 Transformer 语言模型。事实上, BERT 中的许多设计决策都是为了使它们尽可能接近 GPT 而有意做出的, 以便能够对这两种方法进行最小差异的比较。这项工作的核心论点是, 第3.1 节中提出的双向性以及两个预训练任务构成了大部分实证提升的原因, 但我们确实注意到, BERT 和 GPT 在训练方式上还存在以下几处差异:

- GPT 在 BooksCorpus (8 亿词) 上进行训练; BERT 则在 BooksCorpus (8 亿词) 和 Wikipedia (25 亿词) 上进行训练。
- GPT 使用句子分隔符 ([SEP]) 和分类标记 ([CLS]), 但它们仅在微调时才被引入; BERT 则在预训练期间学习了[SEP], [, CLS]以及句子 A/B 嵌入。
- GPT 训练了 1M 步, 批量大小为 32,000 词; BERT 训练了 1M 步, 批量大小为 128,000 词。
- GPT 在所有微调实验中均使用了相同的 5e-5 学习率; 而 BERT 则选择一种在开发集上表现最佳的任务特定微调学习率。

为了隔离这些差异的影响, 我们在第5.1 节中进行了消融实验, 这些实验表明大部分改进实际上来自两个预训练任务及其所实现的双向性。

#### A.5 在不同任务上进行微调的示例

在不同任务上微调 BERT 的图示可参见图 4。我们的特定任务模型是通过将 BERT 与一个额外的输出层相结合而构成的, 因此只需从头学习极少数量的参数。在这些任务中, (a) 和 (b) 是序列级任务, 而 (c) 和 (d) 是词元级任务。在图中,  $E$  表示输入嵌入,  $T_i$  表示词元  $i$ , [CLS] 是用于分类输出的特殊符号, 而 [SEP] 是用于分隔非连续词元序列的特殊符号。

### B 详细的实验设置

#### B.1 GLUE基准测试实验的详细描述。

我们在表 1 中的 GLUE 结果来自 <https://gluebenchmark.com/leaderboard> 和 <https://blog.openai.com/language-unsupervised>。GLUE 基准包括以下数据集, 其描述最初总结于 Wang 等人 (2018a) :

**MNLI** 多体裁自然语言推理是一个大规模、众包的蕴含分类任务 (Williams 等人, 2018)。给定一对句子, 目标是预测第二个句子相对于第一个句子是蕴含、矛盾, 还是中立关系。

**QQP** Quora Question Pairs 是一个二分类任务, 目标是判断在 Quora 上提出的两个问题是否语义等价 (Chen 等人, 2018)。

**QNLI** 问答自然语言推理是斯坦福问答数据集 (Rajpurkar 等, 2016) 的一个版本, 已被转换为二分类任务 (Wang 等, 2018a)。正例是确实包含正确答案的 (问题, 句子) 对, 而负例则是来自同一段落但不包含答案的 (问题, 句子) 对。

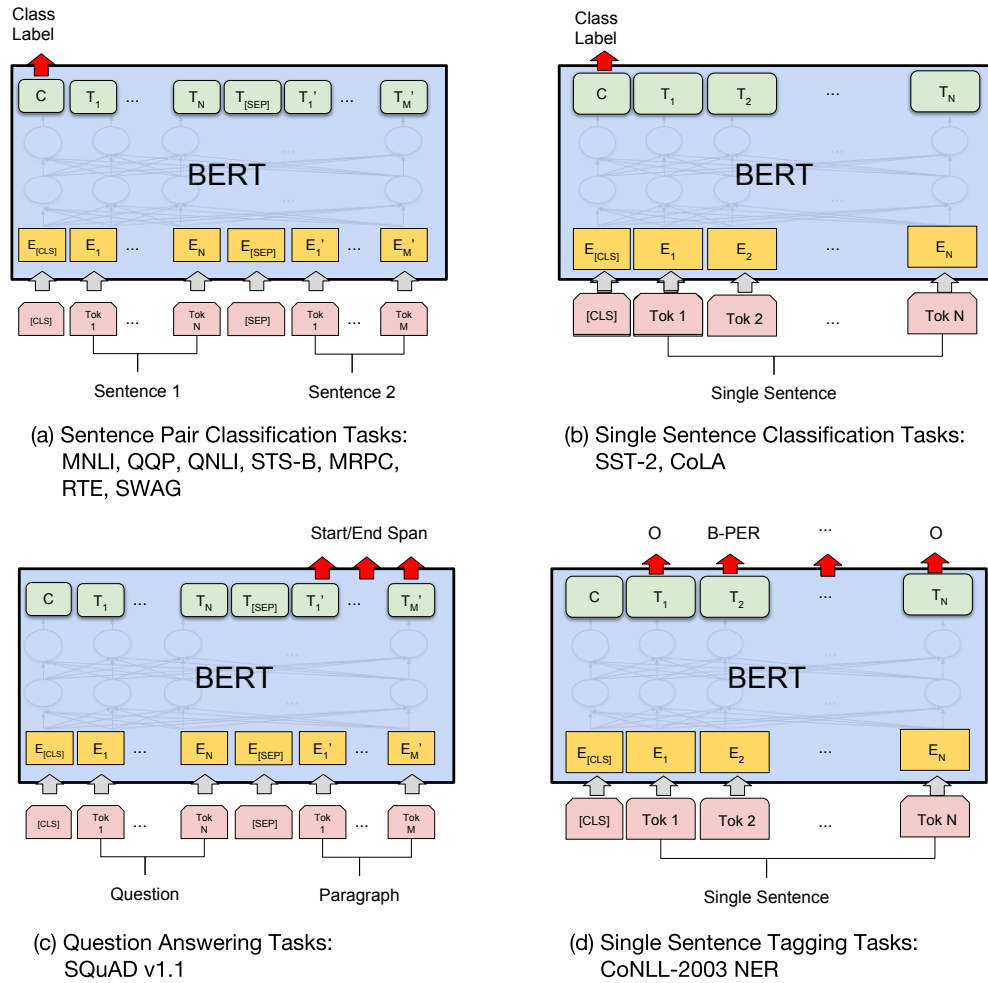


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

**SST-2** The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

**CoLA** The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically “acceptable” or not (Warstadt et al., 2018).

**STS-B** The Semantic Textual Similarity Benchmark is a collection of sentence pairs drawn from news headlines and other sources (Cer et al., 2017). They were annotated with a score from 1 to 5 denoting how similar the two sentences are in terms of semantic meaning.

**MRPC** Microsoft Research Paraphrase Corpus consists of sentence pairs automatically extracted from online news sources, with human annotations

for whether the sentences in the pair are semantically equivalent (Dolan and Brockett, 2005).

**RTE** Recognizing Textual Entailment is a binary entailment task similar to MNLI, but with much less training data (Bentivogli et al., 2009).<sup>14</sup>

**WNLI** Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset,<sup>15</sup> and every trained system that’s been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the ma-

<sup>14</sup>Note that we only report single-task fine-tuning results in this paper. A multitask fine-tuning approach could potentially push the performance even further. For example, we did observe substantial improvements on RTE from multitask training with MNLI.

<sup>15</sup><https://gluebenchmark.com/faq>

**SST-2** 斯坦福情感树库是一个单句二分类任务，由从电影评论中抽取并带有情感人工标注的句子组成 (Socher 等, 2013)。

**CoLA** 语言可接受性语料库是一个二分类的单句分类任务，其目标是预测一个英语句子在语言学上是否“可接受” (Warstadt et al., 2018)。

**STS-B** 语义文本相似度基准 (Semantic Textual Similarity Benchmark) 是由取自新闻标题和其他来源的句子对组成的集合 (Cer 等人, 2017)。这些句子对被标注了从 1 到 5 的分数，表示两个句子在语义含义上的相似程度。

**MRPC** 微软研究释义语料库由从在线新闻源自动提取的句子对组成，并附有人工标注

，用于判断句子对中的句子是否在语义上等价 (Dolan 和 Brockett, 2005)。

**RTE** 识别文本蕴含 (Recognizing Textual Entailment) 是一项与 MNLI 类似的二元蕴含任务，但训练数据要少得多 (Bentivogli 等人, 2009)。<sup>14</sup>

**WNLI** Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset,<sup>15</sup> and every trained system that’s been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the ma-

<sup>14</sup>请注意，本文仅报告单任务微调的结果。多任务微调方法有可能将性能进一步提升。例如，我们确实观察到，通过与 MNLI 进行多任务训练，RTE 上取得了显著的提升。<sup>15</sup><https://gluebenchmark.com/faq>

图4：在不同任务上微调BERT的示意图。

jority class.

## C Additional Ablation Studies

### C.1 Effect of Number of Training Steps

Figure 5 presents MNLI Dev accuracy after fine-tuning from a checkpoint that has been pre-trained for  $k$  steps. This allows us to answer the following questions:

1. Question: Does BERT really need such a large amount of pre-training (128,000 words/batch \* 1,000,000 steps) to achieve high fine-tuning accuracy?  
Answer: Yes, BERT<sub>BASE</sub> achieves almost 1.0% additional accuracy on MNLI when trained on 1M steps compared to 500k steps.
2. Question: Does MLM pre-training converge slower than LTR pre-training, since only 15% of words are predicted in each batch rather than every word?

Answer: The MLM model does converge slightly slower than the LTR model. However, in terms of absolute accuracy the MLM model begins to outperform the LTR model almost immediately.

### C.2 Ablation for Different Masking Procedures

In Section 3.1, we mention that BERT uses a mixed strategy for masking the target tokens when pre-training with the masked language model (MLM) objective. The following is an ablation study to evaluate the effect of different masking strategies.

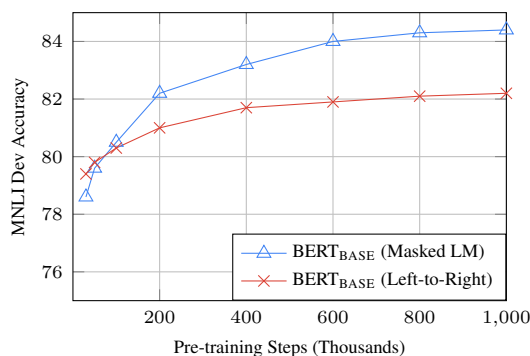


Figure 5: Ablation over number of training steps. This shows the MNLI accuracy after fine-tuning, starting from model parameters that have been pre-trained for  $k$  steps. The x-axis is the value of  $k$ .

Note that the purpose of the masking strategies is to reduce the mismatch between pre-training and fine-tuning, as the [MASK] symbol never appears during the fine-tuning stage. We report the Dev results for both MNLI and NER. For NER, we report both fine-tuning and feature-based approaches, as we expect the mismatch will be amplified for the feature-based approach as the model will not have the chance to adjust the representations.

| Masking Rates |      |      | Dev Set Results   |           |               |
|---------------|------|------|-------------------|-----------|---------------|
| MASK          | SAME | RND  | MNLI<br>Fine-tune | NER       |               |
|               |      |      |                   | Fine-tune | Feature-based |
| 80%           | 10%  | 10%  | 84.2              | 95.4      | 94.9          |
| 100%          | 0%   | 0%   | 84.3              | 94.9      | 94.0          |
| 80%           | 0%   | 20%  | 84.1              | 95.2      | 94.6          |
| 80%           | 20%  | 0%   | 84.4              | 95.2      | 94.7          |
| 0%            | 20%  | 80%  | 83.7              | 94.8      | 94.6          |
| 0%            | 0%   | 100% | 83.6              | 94.9      | 94.6          |

Table 8: Ablation over different masking strategies.

The results are presented in Table 8. In the table, MASK means that we replace the target token with the [MASK] symbol for MLM; SAME means that we keep the target token as is; RND means that we replace the target token with another random token.

The numbers in the left part of the table represent the probabilities of the specific strategies used during MLM pre-training (BERT uses 80%, 10%, 10%). The right part of the paper represents the Dev set results. For the feature-based approach, we concatenate the last 4 layers of BERT as the features, which was shown to be the best approach in Section 5.3.

From the table it can be seen that fine-tuning is surprisingly robust to different masking strategies. However, as expected, using only the MASK strategy was problematic when applying the feature-based approach to NER. Interestingly, using only the RND strategy performs much worse than our strategy as well.

多数类。

## C 补充消融研究

### C.1 训练步数的影响

图5展示了从预训练了  $k$  步的检查点进行微调后的 MNLI 开发集准确率。这使我们能够回答以下问题：

1. 问题：BERT 真的需要如此大量的预训练（128,000 词/批次 \* 1,000,000 步）才能达到较高的微调准确率吗？  
回答：是的，与训练 500k 步相比，训练 1M 步时BERT<sub>BASE</sub> 在 MNLI 上可获得接近 1.0% 的额外准确率。
2. 问题：MLM 预训练是否比 LTR 预训练收敛得更慢，因为每个批次中只有 15% 的词被预测，而不是每个词都被预测？

答：MLM 模型的收敛速度确实比 LTR 模型略慢。然而，就绝对准确率而言，MLM 模型几乎立刻就能开始超越 LTR 模型。

### C.2 不同掩码策略的消融实验

在第3.1节中，我们提到 BERT 在使用掩码语言模型（MLM）目标进行预训练时，采用了一种混合策略来掩码目标词元。以下是一项消融研究，用于评估不同掩码策略的效果。

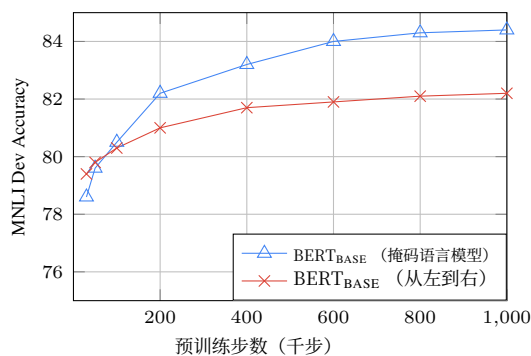


图 5：针对训练步数的消融实验。图中展示了从经过  $k$  步预训练的模型参数出发进行微调后的 MNLI 准确率。横轴为  $k$  的值。

请注意，掩码策略的目的在于减少预训练与微调之间的不匹配，因为 [MASK] 符号在微调阶段从未出现。我们报告了 MNLI 和 NER 两者的开发集（Dev）结果。对于 NER，我们同时报告了微调方法和基于特征方法的结果，因为我们预计不匹配问题在基于特征方法中会被放大，这是由于模型将没有机会调整其表示。

| 掩码率  |     |      | 开发集结果      |      |             |
|------|-----|------|------------|------|-------------|
| 相同掩码 | RND |      | MNLI<br>微调 | 微调   | NER<br>基于特征 |
| 80%  | 10% | 10%  | 84.2       | 95.4 | 94.9        |
| 100% | 0%  | 0%   | 84.3       | 94.9 | 94.0        |
| 80%  | 0%  | 20%  | 84.1       | 95.2 | 94.6        |
| 80%  | 20% | 0%   | 84.4       | 95.2 | 94.7        |
| 0%   | 20% | 80%  | 83.7       | 94.8 | 94.6        |
| 0%   | 0%  | 100% | 83.6       | 94.9 | 94.6        |

表 8：不同掩码策略的消融实验。

结果如表8所示。在该表中，MASK 表示我们在 MLM 中将目标 token 替换为 [MASK] 符号；SAME 表示我们保持目标 token 不变；RND 表示我们将目标 token 替换为另一个随机 token。

表格左侧部分的数字表示MLM预训练期间所使用的特定策略的概率（BERT使用80%、10%、10%）。右侧部分表示开发集结果。对于基于特征的方法，我们将BERT的最后4层拼接起来作为特征，该方法在第 5.3节中被证明是最佳方法。

从表中可以看出，微调对不同遮蔽策略的鲁棒性出人意料地强。然而，正如预期的那样，在将基于特征的方法应用于NER时，仅使用 MASK策略是有问题的。有趣的是，仅使用 RND策略的表现也远不如我们的策略。